Alexandria Engineering Journal 146 (2026) 276–297

Contents lists available at ScienceDirect

Alexandria Engineering Journal

journal homepage: www.elsevier.com/locate/aej

Original article

SMAC-based cooperative hyperparameter optimization for occupational risk assessment in decentralized federated on-device AI networks

Sa Jim Soe Moe [a], Qazi Waqas Khan [a,b], Anh Tuan Nguyen [a], Misbah Bibi [a], Do Hyeun Kim [a,b,*]

[a] Department of Computer Engineering, Jeju National University, Jejusi 63243, Jeju Special Self-Governing Province, Republic of Korea
[b] Big Data Research Center, Jeju National University, Jeju 63243, Republic of Korea

ARTICLE INFO

Keywords:
Hyperparameter optimization
Decentralized federated learning
Construction safety management
On-device AI
Digital twin
Worker safety
Occupational risk assessment

ABSTRACT

Decentralized federated learning (DFL) enables peer-to-peer training, but hyperparameter optimization (HPO) in serverless environments remains underexplored. This paper presents a DFL framework featuring a one-shot cooperative HPO mechanism based on Sequential Model-based Algorithm Configuration (SMAC). The architecture employs a Digital Twin (DT) orchestration layer to manage edge nodes and network topologies without centralizing the learning process. Nodes perform independent local SMAC tuning followed by performance-weighted aggregation to derive a global hyperparameter configuration. Evaluated on worker-safety and MNIST datasets using a Raspberry Pi testbed, our approach significantly outperforms fixed settings and Random Search across IID and non-IID conditions ($p<0.01p < 0.01$ $p<0.01$). Results demonstrate faster convergence and higher accuracy; specifically, convergence rounds dropped from 42 to 18 for worker safety and from 8 to 3 for MNIST. In representative IID mesh experiments, accuracy rose from 92.4% to 95.3%, and MAE improved from 0.058 to 0.042 compared to fixed hyperparameter settings. These findings validate the efficiency of SMAC-based HPO for decentralized, edge-oriented environments.

1. Introduction

Ensuring worker safety is a critical concern in modern industrial environments such as construction sites, manufacturing facilities, and logistics hubs. Predictive modeling can help proactively detect hazardous conditions and prevent accidents, but achieving this in real-time and in the field remains technically challenging. Traditional centralized machine learning approaches are ill-suited for such deployments because of latency, bandwidth restrictions, and privacy issues while sending sensitive data from edge devices to cloud servers. FL offers a promising alternative by enabling multiple clients to collaboratively train a shared model without exchanging raw data [1,2]. However, most FL frameworks follow a centralized architecture, relying on a server to coordinate communication and aggregation. This CFL paradigm introduces bottlenecks related to communication costs, vulnerability to single points of failure, and limited scalability [3–5]. To address these issues, DFL has emerged as a more resilient alternative, where peer nodes train and exchange model updates directly, eliminating the requirement for a central server [6,7]. DFL is especially attractive in use cases involving data heterogeneity, such as healthcare [8], UAV networks [9], and edge-based industrial systems [10–12].

While DFL addresses key infrastructure-level challenges, it presents new ones at the optimization level, particularly in hyperparameter optimization (HPO). In centralized settings, HPO benefits from global visibility; in DFL, each node holds only a small, possibly non-IID subset of data and has limited computational power. Traditional HPO methods, such as grid search or black-box optimization, become impractical due to their resource demands and reliance on repeated global evaluation cycles [13]. Existing federated HPO methods, such as FedEx and FLoRa, remain tailored to CFL settings and do not generalize well to decentralized topologies [14]. Recent research, such as BrainTorrent [15], BlockDFL [16], DFedSAM-MGS [17], and Fedstellar [18], has explored DFL infrastructures. However, HPO in fully decentralized FL remains an underexplored area with only a few recent contributions [19,20]. Notably, while consensus-driven HPO works like DFed-HPO [20] have utilized Bayesian methods, they often rely on iterative peer-voting cycles that increase communication complexity on edge hardware.

To manage the complexity of decentralized training across constrained nodes, we integrate DT technology. Initially proposed by Grieves in 2002 [21], DTs offer synchronized digital replicas of physical systems for real-time control [22]. In our framework, the DT serves as an Orchestration and Observability Plane. It is crucial to distinguish

* Correspondence to: Department of Computer Engineering, Room 423 Jeju National University, Jejusi 63243, Jeju Special Self-Governing Province, Republic of Korea.
E-mail address: kimdh@jejunu.ac.kr (D.H. Kim).

https://doi.org/10.1016/j.aej.2026.05.006
Received 14 August 2025; Received in revised form 4 May 2026; Accepted 6 May 2026
Available online 19 May 2026
1110-0168/© 2026 The Authors. Published by Elsevier B.V. on behalf of Faculty of Engineering, Alexandria University. This is an open access article under the CC BY-NC-ND license (http://creativecommons.org/licenses/by-nc-nd/4.0/).

Nomenclature	
ML	Machine Learning
DT	Digital Twin
CL	Centralized Learning
FL	Federated Learning
CFL	Centralized Federated Learning
DFL	Decentralized Federated Learning
HP	Hyperparameter
HPO	Hyperparameter Optimization
DFL-HPO	Hyperparameter Optimization in Decentralized Federated Learning
SMAC	Sequential Model-based Algorithm Configuration
IID	Independent and Identically Distributed
non-IID	Non-Independent and Identically Distributed
P2P	Peer-to-Peer
IoT	Internet of Things
AI	Artificial Intelligence
AIoT	Artificial Internet of Things
DL	Deep Learning
MAE	Mean Absolute Error
AUC	Area Under the training Curve

this from a central server: the DT does not process raw data or perform model aggregation; rather, it virtualizes the lifecycle of edge nodes to provide administrative control and synchronization [23]. Each edge device is represented as a virtual object, supporting topology configuration (mesh or ring), metadata monitoring, and visualization of learning curves—an essential capability in UI-limited AIoT environments.

In this study, we propose a SMAC-based decentralized HPO mechanism for DFL to enhance occupational risk assessment. SMAC [24] utilizes a Random Forest surrogate that handles mixed-type search spaces more effectively than Gaussian Process-based Bayesian Optimization and scales more efficiently on resource-constrained Raspberry Pis. Our method employs a one-shot cooperative HPO approach: nodes independently optimize hyperparameters locally before performing a single, performance-weighted propagation to derive a global configuration. We validate our approach on both a public classification dataset and a private regression dataset for worker safety. Results show that our method consistently enhances convergence and performance. Statistical analysis confirms that these improvements are significant ($p < 0.01$) compared to both fixed hyperparameter settings and standard Random Search baselines.

Formal problem framing. Let the decentralized federated network be represented by a connected graph $G = (V, E)$, where each node $i \in V$ holds a private local dataset $D_i = D_i^{tr} \cup D_i^{val}$ and exchanges information only with its neighbors. Let $\Lambda = X_1 \times X_2 \times \cdots \times X_d$ denote the mixed hyperparameter search space, where each X_j may be continuous, discrete, or categorical. The decentralized hyperparameter optimization objective is to identify a configuration $\lambda^* \in \Lambda$ that minimizes the average validation loss across nodes without sharing raw data or relying on a central server, i.e.,

$$\lambda^* = \arg\min_{\lambda \in \Lambda} J(\lambda), \qquad J(\lambda) = \frac{1}{|V|} \sum_{i \in V} L\left(w_i(\lambda); D_i^{val}\right).$$

Because $J(\lambda)$ is not directly observable by any individual node in a serverless setting, our framework adopts a two-stage strategy: local SMAC-based tuning at each node followed by one-shot decentralized aggregation of the locally best configurations. The detailed mathematical formulation of the decentralized HPO problem, including the search space, objective function, one-shot dissemination protocol, and performance-weighted aggregation rules, is presented in Sections 3.1 and 3.2.1.

The contributions of the paper are as follows:

1. We introduce a novel framework that embeds a fully decentralized HPO phase, using SMAC-based optimization at each node, into the decentralized federated learning process.
2. We propose a performance-weighted aggregation scheme that derives a global configuration through a one-shot communication phase, minimizing energy consumption on edge devices.
3. We integrate a DT orchestration architecture that virtualizes device management and provides real-time monitoring of decentralized HPO without centralizing the data plane.
4. We demonstrate the effectiveness of our approach on MNIST and a private worker safety dataset using a Raspberry Pi testbed. Experiments confirm that our method achieves superior accuracy and faster convergence ($p < 0.01$) under IID, non-IID, and resource-constrained conditions.

The remainder of the paper is organized as follows: Section 2 presents a summary of related studies. The methodology of the proposed weighted hyperparameters aggregation mechanism is explained in Section 3. Implementation results and performance outcomes are described in detail in Sections 4 and 5. Finally, Section 6 concludes the paper with a discussion of potential directions for future research.

2. Related work and background

Hyperparameters are settings used to control the training process of a machine learning model. Unlike model parameters, which are learned during training, hyperparameters are set before the training process begins and can significantly impact the performance of the model.

2.1. HPO in machine learning and federated learning

HPO is foundational for improving the performance of machine learning models, particularly in scenarios involving complex architectures or sensitive data distributions. Classical techniques such as grid search and random search [25,26] are easy to implement but inefficient in high-dimensional or continuous hyperparameter spaces. Bayesian optimization has gained prominence as a more intelligent approach that models the objective function and guides the search process based on prior evaluations [27]. In federated learning (FL), HPO introduces new challenges. Early studies, such as those by Hsu et al. [28] and Li et al. [2], explored tuning learning rates and local epochs using synthetic or proxy datasets on centralized servers. However, this

introduces discrepancies between validation and real-world data, particularly under non-IID conditions. More advanced HPO methods have been proposed to optimize FL-specific challenges. FedEx [29] utilizes Bayesian optimization and Thompson sampling to coordinate hyperparameter selection across multiple clients, whereas FedTune [30] introduces an adaptive controller for adjusting local epoch and batch sizes. FLoRA [31] offers a single-shot optimization method that pre-selects hyperparameters before model training, reducing communication overhead. Additionally, reinforcement learning–based approaches, such as Auto-FedRL [32] and PSO-based schemes like HPO-PSO [33], show promise; however, all still rely on centralized orchestration for global coordination. These approaches are either impractical or insufficient for fully decentralized environments, where no central server exists to direct HPO decisions.

2.2. Decentralized federated learning architectures

By allowing P2P communication between clients, DFL removes the requirement for a central server. This improves system resilience and privacy while reducing communication bottlenecks. Applications in healthcare [7], mobile edge computing [34], UAV networks [35], and Internet-of-Vehicles [36] have demonstrated the feasibility of DFL under constrained and heterogeneous conditions. To address data and device diversity, Kang et al. [37] introduced blockchain-assisted architectures that ensure privacy-preserving updates, while Li et al. extended this to scalable, incentive-driven systems (e.g., BLADE-FL [9]). Chen et al. [38] proposed mobility-aware learning using multi-agent reinforcement strategies for DFL in vehicular networks, and Li et al. [39] introduced DFedADMM to reduce model inconsistency across asynchronous clients. Recent works, such as Fedstellar [40], present platforms that support both decentralized and semi-centralized modes, introducing features like secure aggregation and asynchronous update scheduling. Yet, these DFL frameworks primarily focus on communication efficiency, model convergence, or data heterogeneity, leaving hyperparameter tuning either unsolved or rigidly pre-defined. Our proposed approach complements these efforts by embedding an HPO mechanism that adapts to local data dynamics without requiring centralized oversight.

2.3. HPO in decentralized federated learning

HPO within DFL remains underexplored due to the challenges posed by serverless architectures and non-IID data. Recent studies have begun addressing these limitations. Wang et al. [41] introduced decentralized bilevel optimization methods focusing on first-order approaches; their work primarily addresses optimization theory rather than direct hyperparameter tuning in federated settings, limiting its direct applicability to DFL-specific HPO. Khan et al. [42] introduced a bilevel HPO strategy for DFL, though it primarily targeted neural architecture search in static ring topologies. Most recently, Khan et al. [43] proposed DFed-HPO, which utilizes HyperOpt with the Adaptive Tree-structured Parzen Estimator (ATPE) for HPO on a Raspberry Pi testbed. Their approach relies on a consensus-driven mechanism where nodes iteratively vote on hyperparameter sets to reach a peer agreement. While DFed-HPO proves that Bayesian-based HPO is feasible in decentralized settings, it relies on iterative consensus, which can introduce significant communication overhead as the number of nodes or the complexity of the search space increases. Furthermore, TPE-based methods like the ATPE model the density of good vs bad configurations, which can be less efficient than the Random Forest surrogate models used in SMAC when handling highly non-linear, noisy objective functions on constrained hardware. Our work distinguishes itself by integrating an SMAC-based tuner within a one-shot cooperative aggregation framework. Unlike iterative consensus, our method performs a single, performance-weighted propagation of local incumbents, drastically reducing communication cycles. Additionally, we integrate a DT orchestration layer to provide real-time observability into the decentralized tuning process, a capability absent in current P2P HPO frameworks.

Among Bayesian HPO strategies, TPE/ATPE-based methods model the density of promising versus non-promising configurations and have already shown feasibility in decentralized edge settings, as reflected by recent consensus-based DFL-HPO studies. In contrast, our method adopts SMAC, whose Random Forest surrogate is better suited to the mixed continuous/discrete/categorical search spaces and low-budget, noisy evaluations considered in this work. We therefore position our contribution not as the first Bayesian HPO method in DFL, but as a communication-efficient one-shot SMAC-based alternative tailored to resource-constrained peer-to-peer learning. Beyond DFL-HPO-specific studies, recent edge-AI works have also addressed demanding application domains such as remote-sensing segmentation [44], structural object detection [45], industrial robot and machinery fault diagnosis [46, 47], and pavement crack detection [48], highlighting the growing need for efficient on-device optimization under heterogeneous sensing and deployment conditions.

2.4. Digital twin–enabled occupational risk assessment

DT systems have progressed from static replicas to real-time orchestrators that fuse IoT sensing, human behavior modeling, and environmental monitoring. In construction and industrial settings, early work by Zhao et al. [49], Akanmu et al. [50], and Somanath et al. [51] couples DTs with wearable sensors and augmented-reality feedback to track posture and site conditions. Fang et al. [52] and Sacks et al. [53] extend this line by introducing DT-based simulations for behavioral-risk assessment and closed-loop safety control. Recent research has begun to marry DTs with distributed learning paradigms. Earlier, Sun et al. [54] integrated adaptive federated learning with DTs for industrial IoT, showing that DT-guided local updates accelerate convergence under device heterogeneity. Kamalakannan et al. [55] propose DTFL-DF, a DT architecture powered by federated learning and decision forests to predict and mitigate underground-mine fire accidents without sharing raw sensor data. Tang et al. [56] develop a DT-enabled, communication-efficient federated learning scheme for vehicular collision warning systems, achieving faster convergence and reduced bandwidth consumption. Although these studies highlight the synergy between DTs and federated analytics, most still rely on a coordinating server or assume homogeneous resources. In contrast, our work tightly couples DT technology with a fully decentralized cooperative HPO pipeline and collaborative learning loop. The DT layer virtualizes each edge node, visualizes the SMAC-based HPO lifecycle, and provides real-time inspection of model configuration and inference results, an essential capability for safety systems, while preserving the privacy-centric, serverless nature of DFL.

2.5. Research gaps

Despite these advancements, a clear gap exists for HPO frameworks that are both computationally efficient and operationally transparent. Current DFL-HPO methods either use iterative consensus logic (e.g., DFed-HPO), which scales poorly in high-latency edge networks, or assume static configurations. Moreover, while Bayesian optimization using TPE has been explored, the use of SMAC, which is better suited for the mixed (categorical and continuous) search spaces found in worker-safety models, has not been applied to fully decentralized federated loops. Finally, most studies focus purely on the optimization logic and neglect the runtime management and monitoring required for industrial AIoT deployments. Our proposed framework fills these gaps by combining SMAC-based local tuning with a lightweight one-shot aggregation rule, all while leveraging DT technology for lifecycle observability. Table 1 summarizes the notable existing works and research gaps addressed in comparison to our work.

Table 1
Summary of notable existing works and research gaps addressed.

Study	FL type	HPO support	Central server	Topology	Edge testbed	DT integrated[a]	Research gap addressed
FedEx [29]	CFL	Bayesian + SHA	Yes	Star	✗	✗	Automated HPO in centralized FL with communication-efficient tuning
Auto-FedRL [32]	CFL	RL-based tuning	Yes	Star	✗	✗	Reinforcement learning-based HPO for CFL scenarios
HPO-PSO [33]	CFL	PSO	Yes	Star	✗	✗	Swarm-based HPO integration in centralized FL
Khan et al. [42]	DFL	Bilevel HPO + NAS	No	Ring	✗	✗	Joint optimization of model architecture and hyperparameters in DFL
Wang et al. [41]	DFL	First-order bilevel	No	Ring	✗	✗	Decentralized bilevel optimization with theoretical convergence
DFed-HPO [43]	DFL	Bayesian (ATPE) + Consensus	No	Mesh	✓	✗	Iterative consensus-based Bayesian HPO under static topology
Ours	DFL	**Bayesian (SMAC) + One-shot**	No	Mesh/Ring	✓	✓	**One-shot cooperative HPO with DT-based observability and edge deployment**

[a] The Digital Twin layer functions as a non-processing observability plane and does not participate in the decentralized model aggregation or hyperparameter optimization logic.

3. Proposed methodology

3.1. Problem setting and formal HPO definition

We consider a connected, undirected peer-to-peer communication graph $G = (V, E)$, where $|V| = N$ denotes the number of edge nodes. Each node $i \in V$ possesses a private local dataset $D_i = D_i^{\text{tr}} \cup D_i^{\text{val}}$. The decentralized HPO search space and objective are defined in Eqs. (1) and (2), respectively, while the clipping rule used to limit the influence of any single node is given in Eq. (3).

To ensure reproducibility (as detailed in Table 4, Section 4), we define the hyperparameter (HP) search space Λ as a multi-domain configuration space:

$$\Lambda = \mathcal{X}_1 \times \mathcal{X}_2 \times \cdots \times \mathcal{X}_d \tag{1}$$

where d is the number of hyperparameters. Each dimension $\mathcal{X}_j$ may be continuous ($\mathbb{R}$), discrete ($\mathbb{Z}$), or categorical ($\mathcal{C}$). A specific configuration is denoted by the vector $\lambda \in \Lambda$.

In a decentralized setting, the goal is to find an optimal configuration λ^* that minimizes the global validation loss without a central orchestrator. The global objective function $J(\lambda)$ is defined as the empirical average of local validation losses:

$$\lambda^* = \arg\min_{\lambda \in \Lambda} J(\lambda), \quad \text{where } J(\lambda) = \frac{1}{N} \sum_{i=1}^{N} \mathcal{L}(w_i(\lambda); D_i^{\text{val}}) \tag{2}$$

where $w_i(\lambda)$ represents the weights learned by node i using configuration λ. Since $J(\lambda)$ is not directly observable by any single node, we employ a two-stage approach: (1) independent local optimization via SMAC, and (2) performance-weighted decentralized aggregation.

Reviewing the weighting mechanism $\alpha_i \propto \exp(-\mathcal{L}_i)$, we recognize that an adversarial node could report a near-zero loss to manipulate the global configuration. To mitigate this, our framework implements weight clipping:

$$\hat{\alpha}_i = \min\left(\alpha_i, \frac{\tau}{N}\right) \tag{3}$$

where τ is a safety threshold (e.g., $\tau = 1.5$). This ensures no single node can disproportionately dominate the decentralized consensus, preserving the integrity of the HPO process against local data noise or malicious reporting.

3.2. Proposed DFL-HPO mechanism

To address the challenges of statistical and system heterogeneity in worker-safety environments, we propose a fully decentralized HPO framework. Our approach prioritizes a node-oriented tuning phase followed by a performance-centric weighting mechanism. Fig. 1 illustrates the overall architecture.

While Bayesian Optimization (BO) is a standard choice for HPO, we specifically utilize SMAC (Sequential Model-based Algorithm Configuration) for three critical reasons:

1. *Handling Mixed Search Spaces:* Unlike Gaussian Process (GP)-based BO, which struggles with non-continuous variables, SMAC uses a Random Forest surrogate that natively handles the mixed-type search spaces (categorical, discrete, and continuous) typical of deep learning architectures.
2. *Scalability:* GPs scale cubically with the number of trials ($O(n^3)$), making them computationally expensive for edge nodes. SMAC's surrogate scales much more efficiently, allowing for higher trial budgets on resource-constrained Raspberry Pi devices.
3. *Robustness to Noisy Landscapes:* Worker-safety sensor data is inherently noisy. SMAC is more robust to high-variance validation losses than Tree-structured Parzen Estimator (TPE) or GP-based methods.

Unlike consensus-based DFL-HPO approaches (e.g., [43]) that require iterative cycles of evaluation and voting to reach an agreement, our method employs a one-shot cooperative aggregation. This minimizes the communication overhead on AIoT devices while ensuring that the final global configuration is informed by the most successful local training landscapes.

3.2.1. Hyperparameter optimization phase

We chose SMAC specifically for DFL due to its unique ability to handle high-dimensional, mixed-type search spaces (categorical, discrete, and continuous), which are common in deep learning architectures. Unlike Gaussian Process-based Bayesian Optimization, SMAC uses a Random Forest surrogate that scales efficiently with the number of trials and is robust to noisy objective functions often encountered on resource-constrained edge devices such as Raspberry Pis.

The HPO phase is designed to be communication-efficient, utilizing a one-shot cooperative approach. This process is divided into local search, global propagation, and decentralized aggregation.

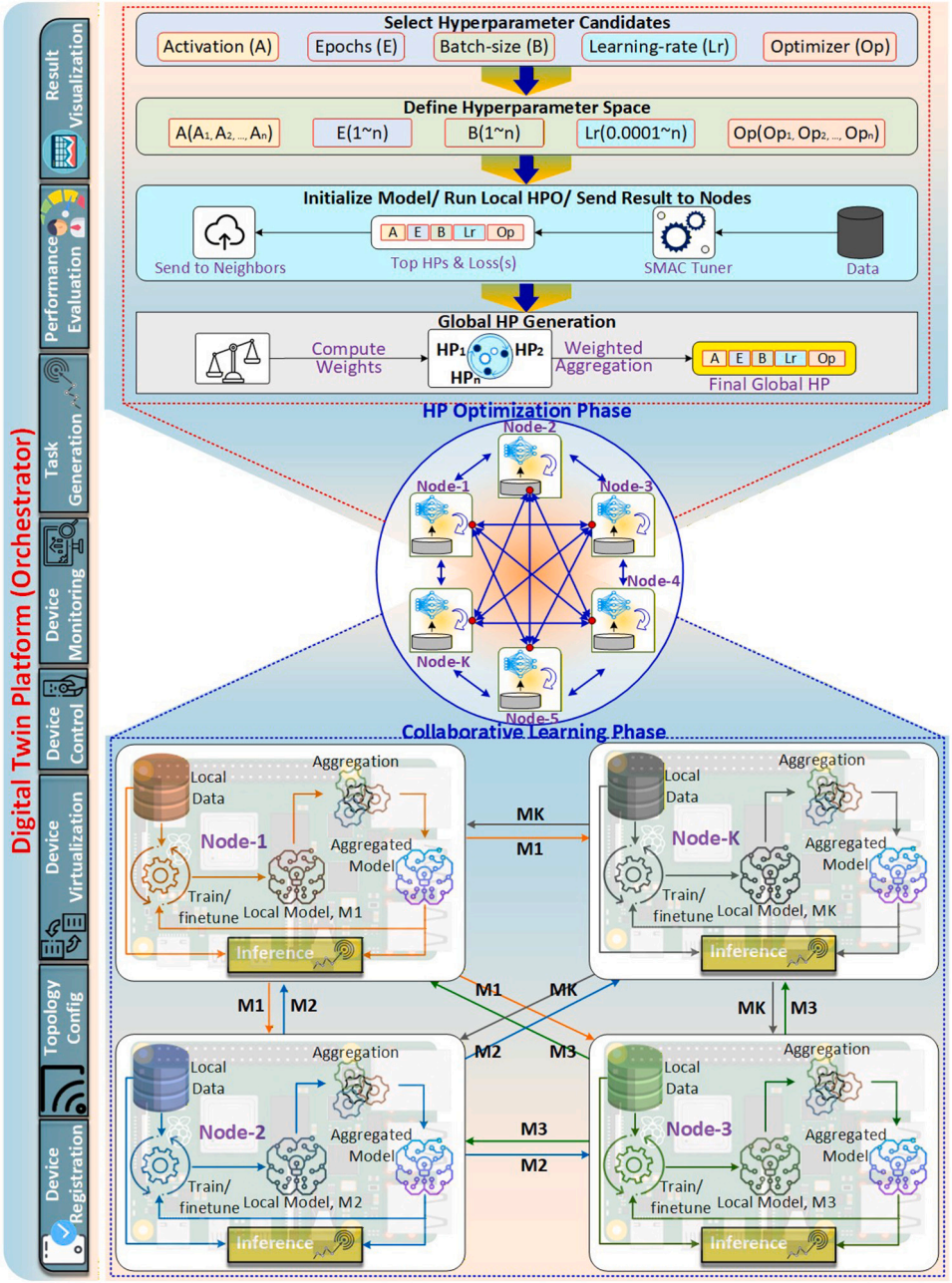


Fig. 1. Proposed overall architecture of SMAC-based HPO in DFL.

1. Local Optimization: Each node $i \in V$ independently executes the SMAC algorithm for B trials. Using a local data subset $\tilde{D}_i$, the node identifies its best-performing local configuration:

$$\lambda_i^* = \arg \min_{\lambda \in \text{Trials}_i} \mathcal{L}(w_i(\lambda); D_i^{\text{val}}) \tag{4}$$

The local incumbent selected by each node during SMAC tuning is given in Eq. (4). The normalized performance signal used during decentralized HPO is defined in Eq. (5), while the exchanged metadata tuple and the reconstructed incumbent set are given in Eqs. (6) and (7), respectively.

Local loss normalization. To address the potential incomparability of validation-loss magnitudes across heterogeneous nodes, each node i normalizes its incumbent validation loss using a local Z-score:

$$\hat{\mathcal{L}}_i = \frac{\mathcal{L}_i^* - \mu_i}{\sigma_i + \delta} \tag{5}$$

where μ_i and σ_i denote the mean and standard deviation of the historical validation losses observed at node i, and δ is a small constant for numerical stability. This normalization ensures that the subsequent weighting reflects relative local performance rather than absolute loss scale.

One-shot dissemination protocol. After completing local SMAC tuning, each node i selects its incumbent hyperparameter configuration λ_i^* together with its corresponding normalized validation loss $\hat{\mathcal{L}}_i$. The node then constructs a metadata-only message

$$M_i = (\lambda_i^*, \hat{\mathcal{L}}_i), \tag{6}$$

which is broadcast to its immediate neighbors. During the HPO phase, nodes exchange only these compact metadata tuples; neither raw training data nor model weights are transmitted. In a connected graph, the received messages are forwarded over peer-to-peer links until graph-wide visibility is achieved. After this one-shot propagation, each node

reconstructs the same incumbent set

$$S = \left\{(\lambda_k^*, \hat{\mathcal{L}}_k)\right\}_{k \in V}, \tag{7}$$

and applies the same deterministic aggregation rules locally. As a result, all nodes obtain an identical global hyperparameter configuration λ_{global} without requiring any central server or coordinator in the learning loop.

2. Communication Flow and Propagation: To address the requirement for a global configuration without a central server, we utilize a single-round propagation (Flooding/Gossip) mechanism.

- *Message Flow*: Each node i broadcasts a tuple $M_i = (\lambda_i^*, \hat{\mathcal{L}}_i)$ to its immediate neighbors $\mathcal{N}(i)$.
- *Termination*: In a connected graph with diameter Δ, messages are forwarded for Δ steps. This ensures that every node i eventually possesses the set of incumbents from the entire network, $S = \{(\lambda_k^*, \hat{\mathcal{L}}_k)\}_{k=1}^N$.
- *One-Shot Nature*: Unlike iterative consensus algorithms (e.g., Average Consensus) that require multiple rounds to converge on a single value, our method exchanges the *final results* of local HPO once.

The aggregation rules for continuous, discrete, and categorical hyperparameters are defined in Eqs. (8)–(11).

3. Performance-Weighted Aggregation: Once a node has collected the set S, it calculates the global configuration λ_{global} locally using the following rules:

Continuous (C): Softmax aggregation. For learning rates or weight decay, we use a softmax-weighted average over normalized validation losses:

$$\lambda_{\text{global}}^{(j)} = \sum_{k=1}^{N} \alpha_k \lambda_{k,j}^*, \qquad \alpha_k = \frac{\exp(-\hat{\mathcal{L}}_k/\tau_s)}{\sum_{m=1}^{N} \exp(-\hat{\mathcal{L}}_m/\tau_s)} \tag{8}$$

where τ_s is a temperature parameter controlling the sharpness of the weighting.

Discrete (D): Rank-based majority voting. For batch size or local epochs, nodes are ranked by normalized validation loss. Let $R(k) \in \{1, \ldots, N\}$ denote the rank of node k in ascending order of $\hat{\mathcal{L}}_k$, and define $w_k = \exp(-\gamma R(k))$. Then:

$$\lambda_{\text{global}}^{(p)} = \arg\max_{v \in S_p} \sum_{k:\, \lambda_{k,p}^* = v} w_k \tag{9}$$

Categorical (G): Shifted inverse-loss voting. Because Z-score normalization may produce negative values, direct inverse-loss voting becomes ill-defined; we therefore apply a minimum-shift to map all normalized losses onto a strictly positive scale. This refinement over a simpler direct inverse-loss formulation ensures the weighting remains well-defined for any Z-score distribution:

$$\tilde{\mathcal{L}}_k = \hat{\mathcal{L}}_k - \min_{m \in V} \hat{\mathcal{L}}_m + \epsilon \tag{10}$$

where $\epsilon > 0$ is a small constant. The categorical hyperparameter is then selected as

$$\lambda_{\text{global}}^{(c)} = \arg\max_{v \in S_c} \sum_{k:\, \lambda_{k,c}^* = v} \frac{1}{\tilde{\mathcal{L}}_k} \tag{11}$$

Scalability of one-shot aggregation. The proposed synchronization scheme is communication-efficient because local tuning and inter-node exchange are decoupled. Each node executes SMAC independently for B trials and communicates only its final incumbent metadata, rather than repeatedly exchanging intermediate candidates during iterative consensus cycles. Hence, the communication burden is not proportional to the local search budget B. In a connected graph, the dissemination stage completes after a bounded number of peer-to-peer propagation steps determined by the graph diameter Δ. This makes the approach more practical for resource-constrained edge devices than iterative peer-voting mechanisms that require repeated synchronization rounds. Nevertheless, as the number of nodes increases, forwarding overhead may still grow with topology density and network size. For this reason, compressed metadata exchange, sparse gossip, and hierarchical aggregation remain promising extensions for larger-scale deployments.

Algorithm 1 Decentralized HPO Phase (executed on each node i)

Input: Local data $\tilde{D}_i$, HP space $\Lambda = C \cup D \cup G$, trials B, clipping threshold τ, temperature τ_s, rank-decay factor γ, stability constants δ, ϵ
Output: Global configuration λ_{global}
Stage 1: Local SMAC Tuning
Initialize surrogate model (Random Forest) and history $\mathcal{H}_i \leftarrow \emptyset$
for trial $s = 1, \ldots, B$ **do**
  $\lambda_i^{(s)} \leftarrow \text{SMAC}(\mathcal{H}_i)$ ▷ Propose next HP configuration
  $\mathcal{L}_i^{(s)} \leftarrow \text{Evaluate}(\lambda_i^{(s)}, \tilde{D}_i)$ ▷ Compute local validation loss
  $\mathcal{H}_i \leftarrow \mathcal{H}_i \cup \{(\lambda_i^{(s)}, \mathcal{L}_i^{(s)})\}$
end for
Identify local incumbent: $\lambda_i^* = \arg\min_s \mathcal{L}_i^{(s)}$, with best loss $\mathcal{L}_i^*$
Normalize local loss: $\hat{\mathcal{L}}_i = (\mathcal{L}_i^* - \mu_i)/(\sigma_i + \delta)$
Stage 2: Decentralized Information Exchange
Broadcast $M_i = (\lambda_i^*, \hat{\mathcal{L}}_i)$ to neighbors $\mathcal{N}(i)$
Propagation: Forward received messages for Δ steps ▷ Ensures global visibility
Form set of all node incumbents $S = \{(\lambda_k^*, \hat{\mathcal{L}}_k) : k \in V\}$
Stage 3: Performance-Weighted Aggregation
1. **Continuous (C):**
  Calculate weights $\alpha_k \propto \exp(-\hat{\mathcal{L}}_k/\tau_s)$
  Apply weight clipping: $\hat{\alpha}_k = \min(\alpha_k, \tau/N)$
  $\lambda_{\text{global}}^{(j)} = \sum \hat{\alpha}_k \lambda_{k,j}^* / \sum \hat{\alpha}_k$
2. **Discrete (D):**
  Rank nodes by $\hat{\mathcal{L}}_k$; calculate $w_k = \exp(-\gamma R(k))$
  $\lambda_{\text{global}}^{(p)} = \arg\max_v \sum_{k:\, \lambda_{k,p}^* = v} w_k$
3. **Categorical (G):**
  Compute shifted scores $\tilde{\mathcal{L}}_k = \hat{\mathcal{L}}_k - \min_m \hat{\mathcal{L}}_m + \epsilon$
  $\lambda_{\text{global}}^{(c)} = \arg\max_v \sum_{k:\, \lambda_{k,c}^* = v} \tilde{\mathcal{L}}_k^{-1}$
Return: $\lambda_{\text{global}} = \{\lambda_{\text{global}}^{(j)} \cup \lambda_{\text{global}}^{(p)} \cup \lambda_{\text{global}}^{(c)}\}$

The detailed logic for this one-shot synchronization is provided in Algorithm 1.

3.2.2. Decentralized federated learning phase

The training phase that follows is illustrated in Algorithm 2, outlining how each node performs local model updates and exchanges weights with its neighbors in a peer-to-peer topology. The decentralized model update, neighbor aggregation, and convergence condition are given in Eqs. (12), (13), and (14), respectively.

This stage describes the DFL process executed at each node k using the globally aggregated hyperparameters λ_{global} obtained from the optimization phase (Algorithm 1). Initially, each node k initializes its local model weights w_k^0 and adopts the shared configuration $\theta_k = \lambda_{\text{global}}$, which includes parameters such as learning rate η, batch size B, number of local epochs E, activation function, and optimizer.

At each communication round r, node k performs local training for E epochs. During each epoch, a mini-batch $\mathcal{B} \subset D_k$ of size B is sampled, and the local gradient g_k is computed using the local loss function $\mathcal{L}(w_k^r, \mathcal{B}, \theta_k)$. The model is then updated using stochastic gradient descent as follows:

$$w_k^r \leftarrow w_k^r - \eta \cdot g_k \tag{12}$$

Upon completion of local training, node k communicates its current model w_k^r with its immediate neighbors N_k and performs decentralized

Algorithm 2 Decentralized Federated Learning (DFL) at Node k

Input: Global hyperparameters λ_{global}, local dataset D_k, total rounds R, neighbors N_k, learning rate η, mini-batch size B, local epochs E
Output: Converged local model weights w_k^R
Initialize local model weights w_k^0 and set $\theta_k \leftarrow \lambda_{\text{global}}$
for round $r = 0, \ldots, R-1$ **do**
  Local Training at node k**:**
  for local epoch $e = 1, \ldots, E$ **do**
    Sample mini-batch $\mathcal{B} \subset D_k$ of size B
    Compute gradient $g_k \leftarrow \nabla \mathcal{L}(w_k^{(r)}, \mathcal{B}, \theta_k)$
    Update weights: $w_k^r \leftarrow w_k^r - \eta \cdot g_k$
  end for
  Decentralized Aggregation:
  Exchange w_k^r with neighbors $\{w_i^r\}_{i \in N_k}$
  Aggregate weights:

$$w_k^{(r+1)} \leftarrow \frac{1}{|N_k|+1}\left(w_k^r + \sum_{i \in N_k} w_i^r\right)$$

  Convergence Check:
  if $\|w_k^{(r+1)} - w_k^r\| \le \epsilon$ **then**
    break
  end if
end for
Return: w_k^R

model aggregation through weighted averaging:

$$w_k^{(r+1)} \leftarrow \frac{1}{|N_k|+1}\left(w_k^r + \sum_{i \in N_k} w_i^r\right) \tag{13}$$

This fully decentralized exchange and aggregation process ensures collaborative training without reliance on a central server. After each round, node k checks for convergence by evaluating the norm of the model update:

$$\left\|w_k^{(r+1)} - w_k^r\right\| \le \epsilon \tag{14}$$

If the condition is satisfied, the node halts training; otherwise, it proceeds to the next round.

At the end of R rounds, each node obtains its personalized model weights w_k^R, which embed both locally learned patterns and information shared by neighboring nodes, making them suitable for on-device inference tasks such as real-time occupational risk assessment.

3.3. Digital twin orchestration and observability plane

Our framework utilizes a DT layer as a control and observability plane, while the DFL nodes constitute the data plane. Unlike a centralized federated learning server, the DT does not collect raw training data, execute local model training, or perform decentralized weight aggregation. Instead, it virtualizes the lifecycle of the edge nodes and provides system-level services required to deploy, monitor, and manage decentralized HPO and DFL in a practical AIoT environment.

Fig. 2 presents the functional mapping of the DT orchestration plane. The DT first supports device registration, where edge devices are enrolled using network descriptors such as IP address and port. It then provides topology configuration, enabling the peer-to-peer communication structure to be organized as mesh or ring before decentralized execution begins. Through device virtualization, the DT maintains a digital representation of each node, including status and device information, allowing the system to monitor node availability and runtime conditions in real time.

Table 2
Hardware and software configuration of Digital Twin (Orchestrator).

Environments	Digital Twin platform (Orchestrator)
Hardware	
Operating system	Windows 10 Pro 64-bit
CPU	Intel® Core™ i9-11900F 2.5 GHz
Memory	64 GB RAM
Disk storage	1000 GB SSD
Software	
Programming language	Python 3.8
Development environment	PyCharm IDE
Deep learning framework	TensorFlow 2.6, Keras 2.6
Web framework	Flask 2.2.2
Database	MySQL (Workbench 8.0)
Libraries	NumPy 1.19.5, Requests 2.28

The DT also supports model management and task generation. In this stage, the model structure and execution settings for both DFL and HPO are configured before runtime, including the selected dataset, number of communication rounds, local epochs, and task type. In addition, the DT handles data management only at the metadata level, such as device descriptors, performance logs, and exported monitoring records. This means that the DT manages lightweight control-plane information rather than raw sensor data or model parameters.

During runtime, the DT provides device monitoring and device control. The monitoring function tracks node connectivity and runtime performance indicators, while the control function enables the operator to activate or deactivate devices and trigger HPO or DFL-related processes. Finally, the result visualization module displays the outcomes of decentralized execution, including local/global hyperparameter configurations, device-level training performance, and worker-safety inference results. This mapping clarifies that the DT is not a conceptual overlay, but a concrete orchestration layer that improves coordination, traceability, and operational transparency in decentralized edge learning.

From the system-overhead perspective, the DT introduces a significantly lighter burden than a centralized training server because it does not aggregate model updates or process the full learning workload. Its role is restricted to control-plane signaling, metadata management, and visualization. Therefore, the additional overhead of the DT is primarily associated with registration, topology setup, telemetry handling, and interface rendering, while the computationally intensive training and inference tasks remain on the edge devices. The quantitative hardware and communication implications of this design are discussed in Section 5.3.

4. Experiment and implementation results

4.1. Experimental setup

Table 2 presents the hardware and software configuration used for developing and deploying the DT platform, which acts as the observability platform in this work. The system runs on a high-performance PC to manage global coordination and database operations. Development was performed using Python 3.8, with TensorFlow 2.6 and Keras 2.6 serving as the primary deep learning frameworks.

Table 3 details the configuration for each on-device AI node. Each DFL client is implemented on a Raspberry Pi 4 running Ubuntu 20.04. These nodes utilize EdgeX Foundry and Docker for managing containerized microservices, enabling decentralized intelligence at the edge. While the integration of EdgeX Foundry and Flask introduces a software layer for orchestration, the resource overhead is minimized through containerization and functional decoupling. EdgeX Foundry serves as a lightweight middleware specifically optimized for ARM-based edge gateways; in our safety-critical scenario, it handles low-bandwidth telemetry data (sensor readings) rather than high-throughput streams,

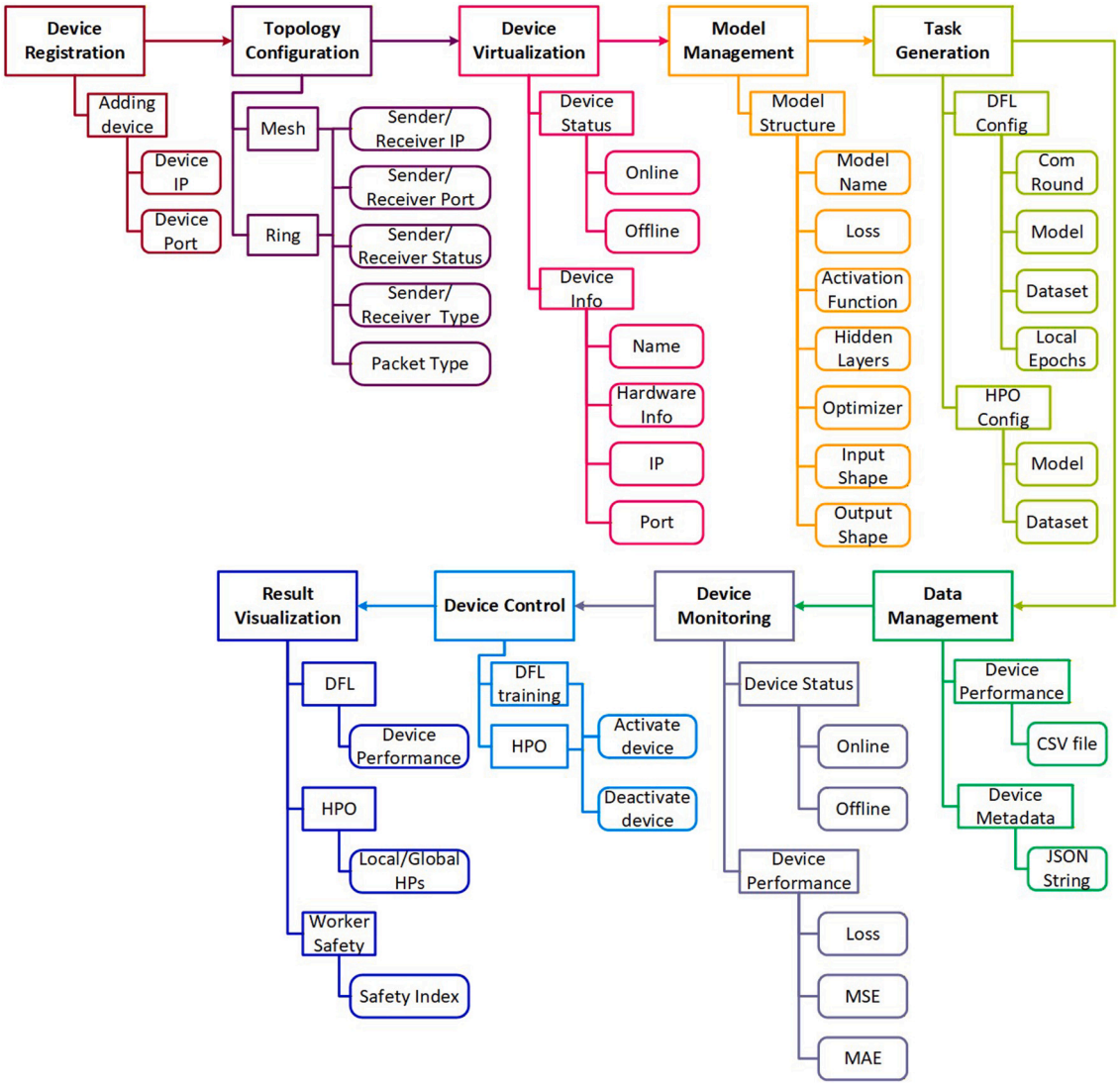


Fig. 2. Functional mapping of the Digital Twin (DT) orchestration plane. The DT layer supports device registration, topology configuration, virtualization, monitoring, control, metadata handling, and result visualization, while HPO, local training, and decentralized model aggregation remain on the edge nodes.

resulting in negligible CPU idling. Furthermore, the Flask-based DT application on the DFL node primarily manages control-plane signals (e.g., node registration and HPO coordination), ensuring that the bulk of the 4 GB RAM and quad-core CPU remain available for the compute-intensive TensorFlow training and inference processes.

Network setup and deployment workflow. The experimental platform consisted of six Raspberry Pi 4 nodes and one PC hosting the DT interface, all connected to the same local area network through a wireless router. Peer-to-peer communication among the Raspberry Pi nodes was implemented using REST/HTTP endpoints exposed via Flask, while EdgeX Foundry was used locally at the edge to register IoT devices and collect sensor streams. The DT remained on the PC side and operated only in the control/observability plane. Although all devices shared the same LAN, the communication topology was enforced logically in software: in the mesh setting, each node was allowed to communicate with all designated peers, whereas in the ring setting each node exchanged messages only with its two assigned neighbors. The end-to-end execution flow consisted of: (1) node registration, (2) topology assignment, (3) HPO configuration broadcast, (4) local SMAC execution, (5) one-shot hyperparameter propagation, (6) global hyperparameter reconstruction, (7) synchronized DFL training start, (8) per-round model exchange and aggregation, and (9) inference/visualization. Unless otherwise stated, each experiment was executed for 50

Table 3
On-device AI node (DFL Client) configuration.

Environments	DFL Node (Raspberry Pi 4)
Hardware	
Operating system	Ubuntu 20.04 64-bit
Device	Raspberry Pi 4
CPU	Quad-core 64-bit SoC @ 1.5 GHz
Memory	4 GB RAM
Disk storage	32 GB microSD card
Software	
Programming language	Python 3.8
Deep learning framework	TensorFlow 2.6, Keras 2.6
Edge/IoT platform	EdgeX Foundry, Docker, Docker Compose
Supporting libraries	NumPy 1.19.5, Requests 2.28, smac 2.3

Table 4
Hyperparameter search space for MNIST and Safety datasets.

Hyperparameter	Type	Range/Values	Applied to
Learning rate	Continuous	$[10^{-4}, 10^{-1}]$ (log-uniform)	Both Datasets
Batch size	Discrete	$\{32, 64, 128\}$	MNIST
Batch size	Discrete	$\{16, 32, 64\}$	Safety
Local Epochs (E)	Discrete	$\{2, 4, 6, 8\}$	Both Datasets
Optimizer	Categorical	{SGD, Adam}	Both Datasets
Activation	Categorical	{ReLU, Tanh}	Both Datasets

Table 5
Optimal hyperparameter configurations on a node for IID and non-IID scenarios.

Hyperparameter	MNIST		Safety	
	IID	non-IID	IID	non-IID
Learning rate	0.01	0.005	0.001	0.0008
Batch size	64	32	32	16
Local Epochs (E)	4	8	4	6
Optimizer	Adam	Adam	SGD	SGD
Activation	ReLU	ReLU	ReLU	Tanh

communication rounds, and aggregation was performed synchronously after each node received the required updates from its neighbors.

The hyperparameter search space for the experimental evaluations is summarized in Table 4. We utilized a log-uniform distribution for the learning rate to ensure exhaustive coverage across multiple scales. Notably, the batch size for the private Safety dataset was constrained to a smaller range ($\{16, 32, 64\}$) compared to the standard MNIST range to better accommodate the specific dimensionality and sample density of the tabular Safety data. To respect the Raspberry Pi memory limits and ensure stable execution within the containerized environment, the maximum batch size across all experiments was capped at 128.

Following the optimization process, the specific configurations that yield the best performance on a node in both IID and non-IID scenarios are presented in Table 5. The tuning results indicate that non-IID distributions generally require smaller batch sizes and a higher number of local epochs to mitigate the weight divergence inherent in heterogeneous edge data. Additionally, while the MNIST experiments leveraged the Adam optimizer, the private tabular Safety dataset achieved better convergence stability using SGD, reflecting the different optimization requirements between image-based and tabular-regression tasks at the edge.

Fig. 3 illustrates the physical client devices used in the experimental evaluation of this study. Raspberry Pi 4B platforms are utilized as DFL nodes, endowed with machine learning capabilities to support both model training and inference. In this experimental setup, six Raspberry Pi 4B devices are deployed to construct a testbed primarily dedicated to hyperparameter optimization, followed by model training using a worker safety dataset within a DFL environment. The use of actual physical hardware, rather than purely simulated clients, is intended to provide a realistic experimental setting, which is essential to ensuring the validity and reliability of the research findings.

4.2. Implementation results

This subsection describes the implementation results of the HPO and occupational risk assessment on DT, which serves all operations to ensure a smooth process through its virtualization and visualization capabilities.

4.2.1. Hyperparameter optimization and observability via DT

In fully decentralized environments, the absence of a central server can lead to black-box training where node-level failures or suboptimal configurations go unnoticed. We incorporate a DT layer to serve as a secure Orchestration and Observability Plane. It is crucial to distinguish this from a central server: the DT does not process raw data or perform model aggregation; rather, it virtualizes the lifecycle management of edge nodes to provide high-level administrative control.

As shown in Fig. 4, the DT platform provides a web-based interface for node registration and HPO monitoring. Once the user defines the network topology (Mesh or Ring), the DT acts as a control-plane trigger to initiate the local SMAC tuning.

To preserve the decentralized integrity of the system, the DT-led HPO process is a one-shot task. The DT monitors the logs (as shown in Fig. 5) as the nodes communicate peer-to-peer to aggregate their hyperparameter incumbents. This architecture ensures that the computational and privacy-centric benefits of DFL are maintained, while the DT provides the transparency required for safety-critical industrial deployments.

4.3. Occupational risk assessment

After completing the DFL, the trained model, which has been enhanced by aggregating knowledge from neighboring sources, is used to predict outcomes using unseen data from IoT devices equipped with sensors. To facilitate the prediction process for worker safety, DT creates a virtual replica of each worker along with a corresponding monitoring panel to display the safety index. The safety index ranges from 1 to 10, where a score of 1 to 4 indicates the safest conditions, 5 to 7 signifies caution, and 8 to 10 represents dangerous conditions. On the DT interface, the worker's safety index is visually represented with colors ranging from green (safe) to red (danger). Fig. 6 illustrates the user interface for occupational risk assessment on the DT. The performance of the model on each node is visualized on DT for tracking and evaluation. On the right side of the UI, when the "Start Inference" button is clicked, the monitoring results for the safety of each worker become visible.

Fig. 7 depicts the step-by-step inference process used on the node to enable real-time occupational risk assessment. In this setup, sensor data from outdoor construction sites is gathered every 15 s. This data includes structural metrics (strain, load, inclinometer), environmental factors (wind, temperature, precipitation), and worker-specific information (heart rate, GPS location). It is transmitted via EdgeX microservices and stored locally. After preprocessing, the test data is processed by the inference model on each edge device. The model generates a Safety Index that indicates each worker's predicted safety level in near real-time. This approach allows continuous, low-latency safety monitoring and prediction directly at the site, without relying on centralized processing, making it ideal for dynamic outdoor construction environments.

5. Performance evaluation

5.1. Evaluation setup and metrics

To evaluate the effectiveness of the proposed DFL-HPO framework, we utilize two distinct learning tasks: a real-world worker safety regression dataset and the MNIST image classification benchmark. For all experiments, data is distributed across six simulated edge nodes. We evaluate performance under two conditions: (1) an IID setting where data is shuffled and distributed uniformly, and (2) a non-IID setting

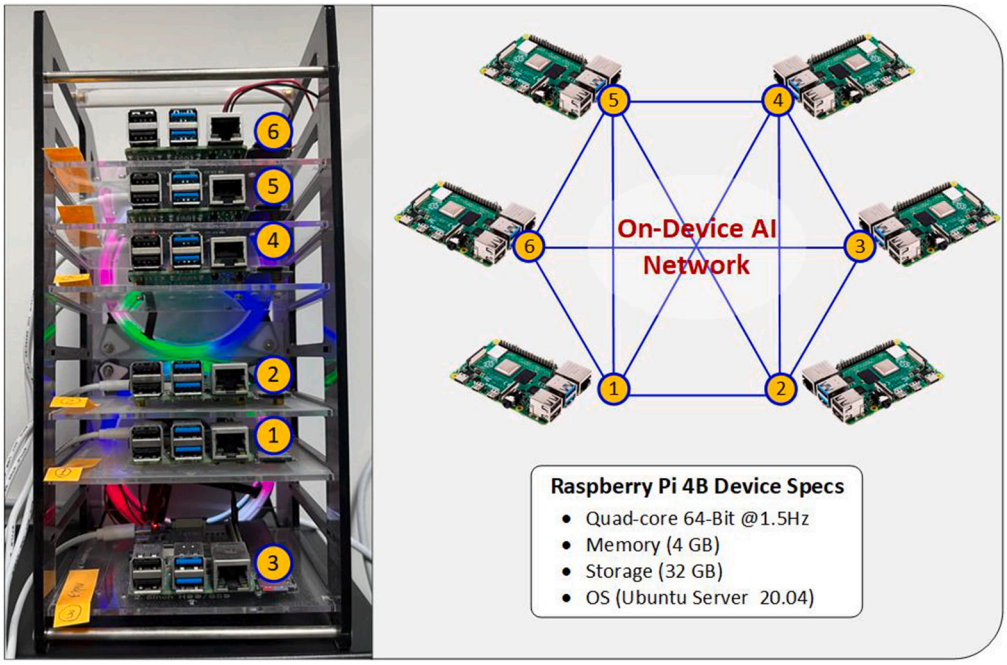


Fig. 3. Physical testbed used to form on-device AI network.

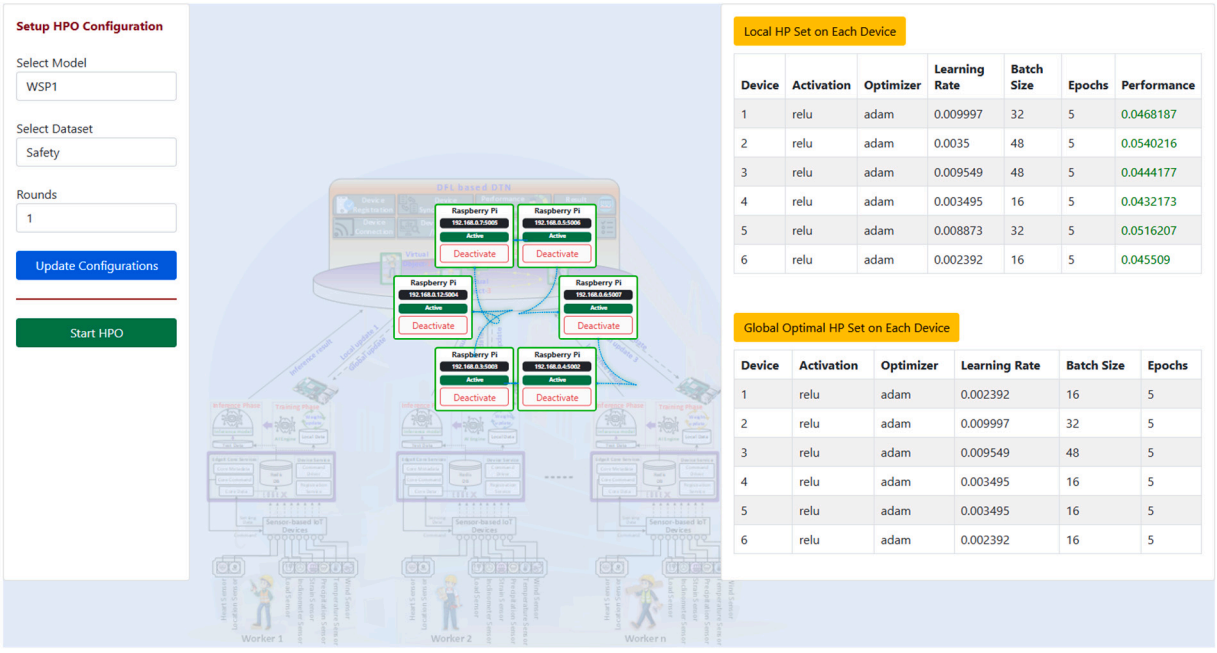


Fig. 4. User interface for HPO phase on DT.

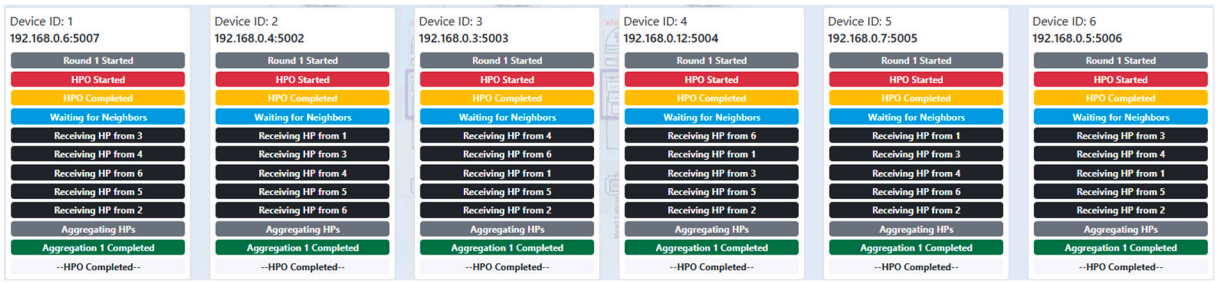


Fig. 5. One-shot cooperative HPO on DT.

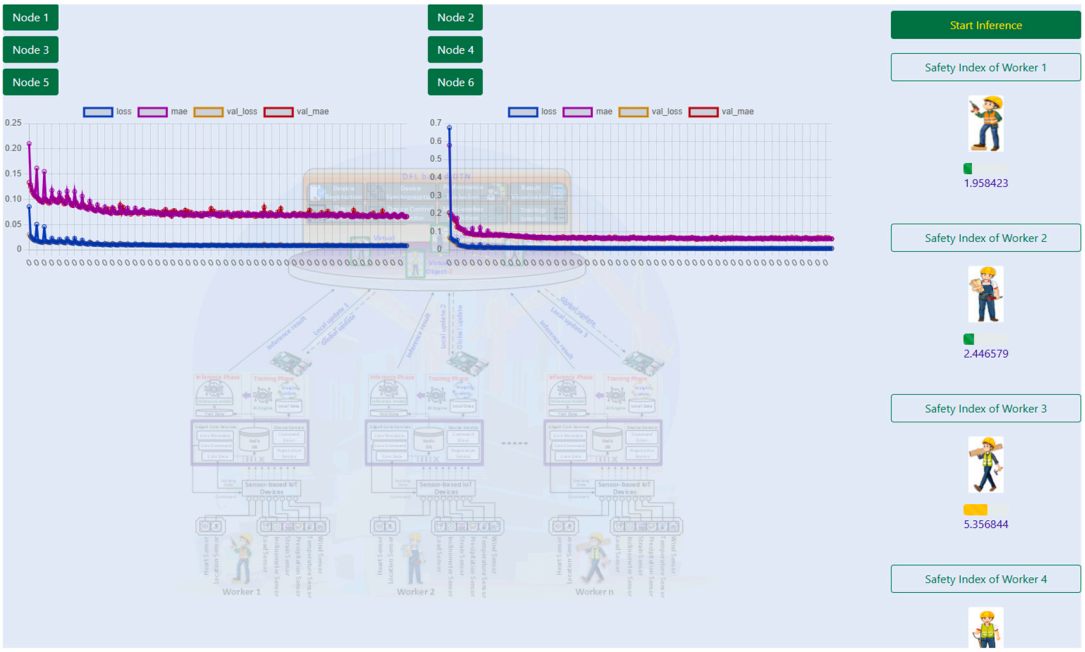


Fig. 6. Occupational risk assessment UI on DT.

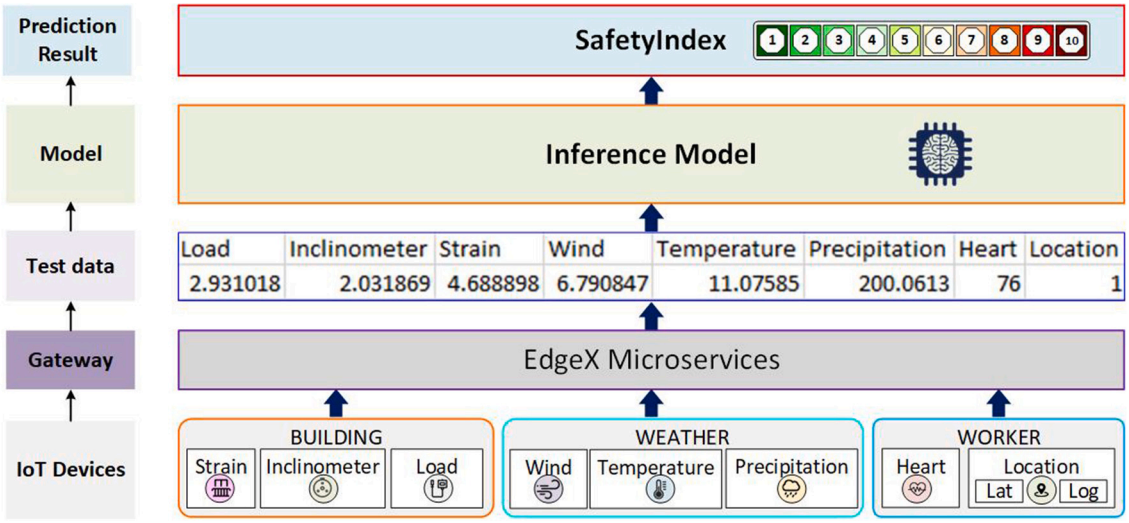


Fig. 7. Occupational risk assessment process on the node.

designed to reflect the data heterogeneity inherent in decentralized environments.

The worker safety dataset, sourced from a research facility at Jeju National University, consists of 50,000 records. As illustrated in Fig. 8, the dataset is derived from raw sensor inputs through a structured data processing and feature selection pipeline. Each sample integrates eight distinct input features categorized as follows:

- **Environmental conditions:** Wind velocity (m/s), precipitation levels (mm), and ambient temperature (°C).
- **Structural indicators:** Mechanical load (kN), structural strain (με), and object inclination (°).
- **Physiological and spatial data:** Worker heart rate (bpm) and GPS-derived proximity to hazardous zones (m).

The regression target is a continuous Safety Index, representing a normalized risk assessment score. To develop a realistic heterogeneous regression scenario, we divided the continuous Safety Index target into ten bins based on quantiles. For each bin, we sampled a client-allocation probability vector $\mathbf{p}$ from a Dirichlet distribution with concentration parameter $\alpha = 0.5$, and then distributed the samples in that bin across the six clients according to $\mathbf{p}$. This procedure yielded highly skewed client-level target distributions while preserving the overall global statistics of the dataset. As a result, each node observed a distinct proportion of low-, medium-, and high-risk samples, which directly altered the local validation landscape explored during SMAC tuning and influenced the subsequent convergence behavior of decentralized training.

For comparison, we also utilize the MNIST dataset, which contains 70,000 28 × 28 grayscale images of handwritten digits (60,000 for training and 10,000 for testing). Similar to the worker safety dataset, MNIST is partitioned using the same Dirichlet-based non-IID strategy to ensure a consistent evaluation of the DFL-HPO framework across both regression and classification tasks.

To ensure reproducibility and quantify variance, the local SMAC budget was fixed to 8 trials for MNIST and 5 trials for the Worker

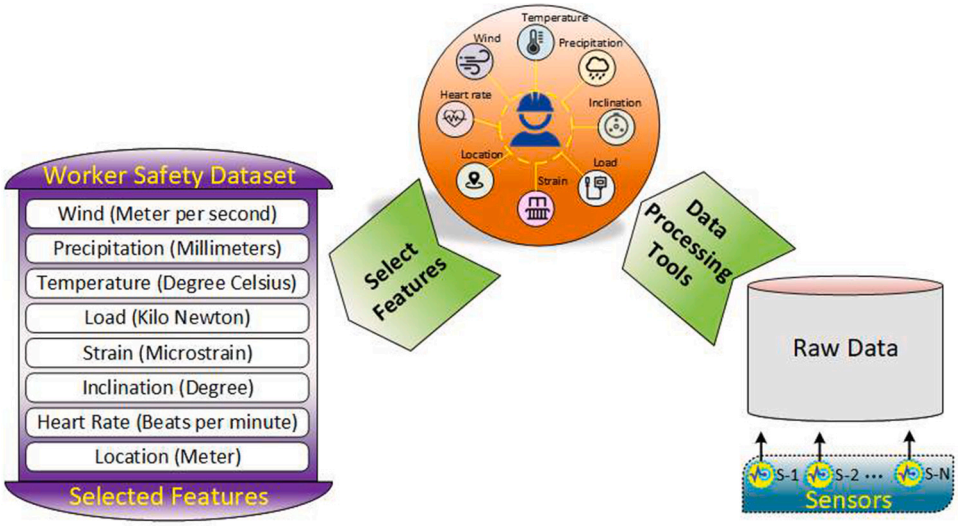
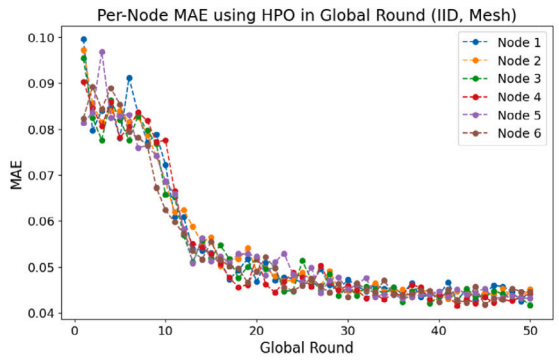
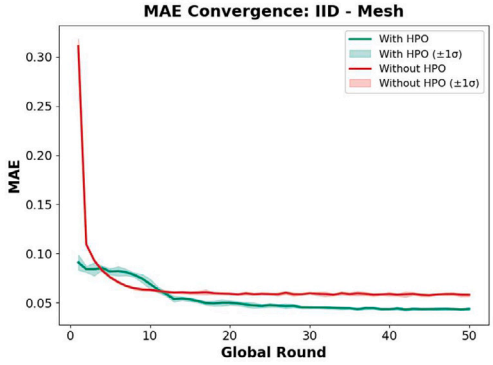


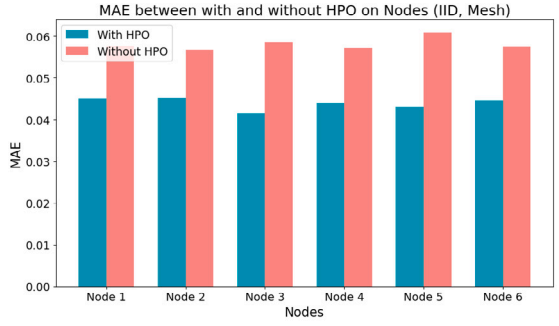
Fig. 8. Overview of the worker safety data acquisition process.



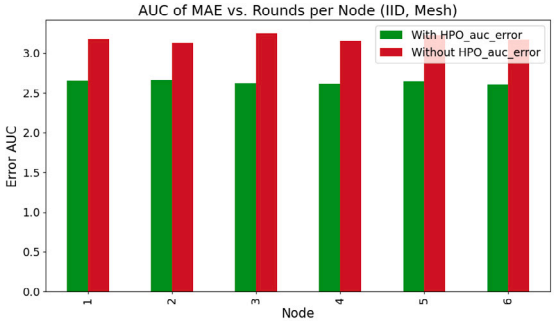
(a) Per node MAE across rounds using HPO in mesh network



(b) Per-round average MAE comparison in mesh network: with vs without HPO



(c) Per node MAE comparison in mesh network: with vs without HPO

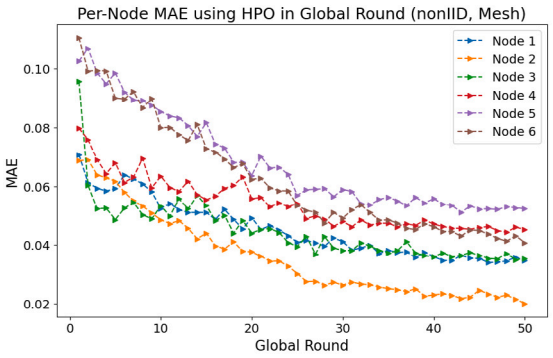


(d) AUC of MAE per node comparison in mesh network: with vs without HPO

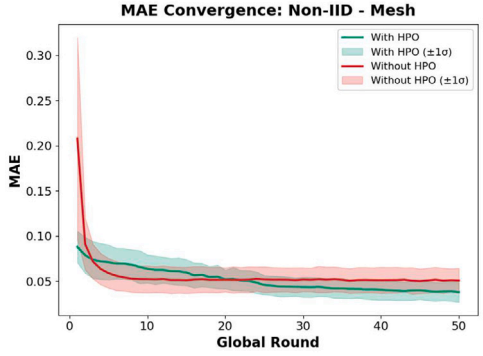
Fig. 9. Decentralized training results under IID mesh settings using the Worker Safety dataset: (a) per-node MAE across rounds using HPO, (b) average MAE per round comparing with and without HPO with shaded regions indicating $\pm 1\sigma$ variance across five independent runs, (c) final per-node MAE comparison, and (d) AUC of MAE per node comparison for learning stability assessment.

Safety dataset, and all experiments were repeated over 5 independent runs with different random seeds to quantify variance across stochastic HPO, SGD updates, and non-IID data sampling. In convergence plots Figs. 9(b), 10(b), 11(b), 12(b), 13(a), 14(a), 15(a), 16(a), solid lines show mean performance while shaded regions represent ± 1 standard deviation, demonstrating the stability of our approach.

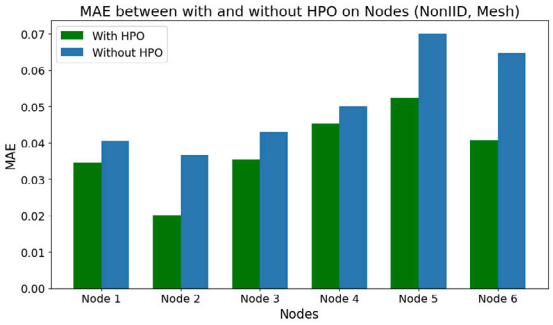
In this study, we employed several statistical evaluation metrics to assess model performance. For regression tasks, we employed the Mean Absolute Error (MAE) (15), while for classification tasks, we focused on Accuracy (16). Additionally, to evaluate the effectiveness of our approach for HPO, we included two convergence-related metrics: the AUC and the number of rounds to reach a predefined performance threshold. The AUC in this context measures the integral of the performance curve (accuracy (17) or error (18)) over the number of global communication rounds, capturing both the speed and stability of convergence. We note that this is not the AUC-ROC used in classification analysis, but rather a temporal measure of cumulative performance.



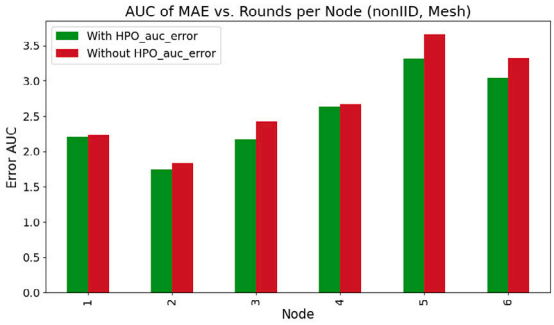
(a) Per node MAE across rounds using HPO in mesh network



(b) Per round average MAE comparison in mesh network: with vs without HPO



(c) Per node MAE comparison in mesh network: with vs without HPO



(d) AUC of MAE per node comparison in mesh network: with vs without HPO

Fig. 10. Decentralized training results under non-IID mesh settings using the Worker Safety dataset: (a) per-node MAE across rounds using HPO, (b) average MAE per round comparing with and without HPO with shaded regions indicating $\pm 1\sigma$ variance across five independent runs, (c) final per-node MAE comparison, and (d) AUC of MAE per node comparison for learning stability assessment.

Mean absolute error (MAE). MAE quantifies the average magnitude of the absolute errors between actual and predicted values:

$$\text{MAE} = \frac{1}{n}\sum_{i=1}^{n}\left|Y_i - \hat{Y}_i\right| \tag{15}$$

Accuracy. Accuracy is defined as the ratio of correct predictions to all predictions:

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN} \tag{16}$$

Area under the accuracy–round curve (AUC_ACC). Let a_r denote the accuracy achieved at round r, and R be the total number of rounds. The AUC of the accuracy curve is calculated as:

$$\text{AUC}_{\text{ACC}} = \sum_{r=1}^{R-1}\frac{a_r + a_{r+1}}{2}\cdot \Delta r \tag{17}$$

where $\Delta r = r_{i+1} - r_i$ (typically $\Delta r = 1$). A higher AUC_{ACC} indicates faster attainment of high accuracy and its sustained maintenance throughout training.

Area under the MAE–round curve (AUC_MAE). For regression tasks, we analogously define the area under the MAE–round curve to capture convergence speed in terms of error reduction. Let e_r denote the MAE at round r. The AUC for MAE is given by:

$$\text{AUC}_{\text{MAE}} = \sum_{r=1}^{R-1}\frac{e_r + e_{r+1}}{2}\cdot \Delta r \tag{18}$$

A lower AUC_{MAE} indicates that the model reached low error values earlier and maintained them over the remaining rounds, reflecting faster and more stable convergence.

Rounds to threshold performance. For classification, this metric records the first global communication round r at which the validation accuracy reaches or exceeds a specified threshold (e.g., 90%). If the threshold is never reached, the metric is recorded as undefined (NaN).

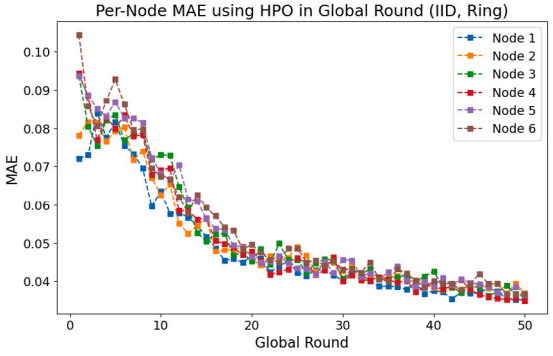
5.2. Analysis results

5.2.1. Performance evaluation using worker safety data

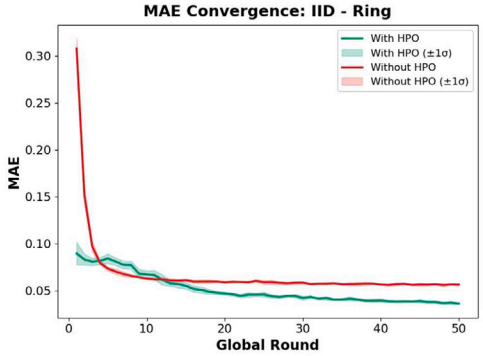
Analysis results in mesh topology.

IID scenario: Fig. 9 illustrates the performance of decentralized learning with and without hyperparameter optimization (HPO) using the Worker Safety dataset under IID distribution and a mesh network topology. As shown in Fig. 9(a), the mean absolute error (MAE) of each node decreases steadily across global communication rounds when using HPO, reflecting consistent local improvements. Fig. 9(b) further demonstrates that HPO leads to lower average MAE over rounds compared to training without HPO, indicating better global predictive performance. In Fig. 9(c), per-node MAE results reveal that all six nodes benefit from HPO, achieving significantly lower error values. Lastly, Fig. 9(d) presents the area under the MAE curve (AUC), where lower AUC values with HPO imply enhanced learning stability and reduced prediction error across nodes. These results confirm that applying HPO not only improves convergence and prediction accuracy but also enhances consistency across the network in regression tasks under IID conditions.

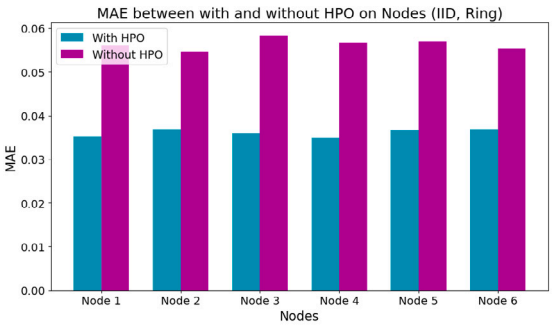
Non-IID scenario: Fig. 10 illustrates the performance of decentralized regression training in a non-IID mesh network using the Worker Safety dataset, comparing models trained with and without HPO. Fig. 10(a) presents the per-node MAE across global rounds, where HPO consistently reduces the prediction error across all nodes as training progresses. In Fig. 10(b), the average MAE over communication rounds



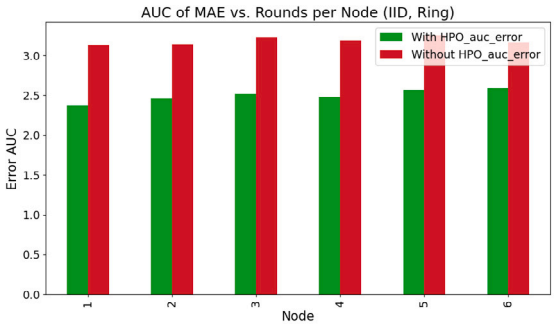
(a) Per node MAE across rounds using HPO in ring network



(b) Per round average MAE comparison in ring network: with vs without HPO



(c) Per node MAE comparison in ring network: with vs without HPO



(d) AUC of MAE per node comparison in ring network: with vs without HPO

Fig. 11. Decentralized training results under IID ring settings using the Worker Safety dataset: (a) per-node MAE across rounds using HPO, (b) average MAE per round comparing with and without HPO with shaded regions indicating $\pm 1\sigma$ variance across five independent runs, (c) final per-node MAE comparison, and (d) AUC of MAE per node comparison for learning stability assessment.

is significantly lower with HPO, especially in early rounds, indicating faster convergence and better generalization. Sub Fig. 10(c) further confirms that nodes trained with HPO achieve lower final MAE values compared to those without HPO. Importantly, the corrected sub Fig. 10(d) shows the area under the MAE curve (AUC) per node, where HPO results in smaller AUC values across all nodes, signifying improved learning stability and more efficient error reduction over time. This analysis reinforces the value of hyperparameter optimization in achieving robust performance under data heterogeneity in decentralized regression tasks.

Analysis results in ring topology.

IID scenario: Fig. 11 presents the performance evaluation of decentralized training with and without HPO under IID ring topology using the Worker Safety dataset. Sub Fig. 11(a) illustrates the per-node MAE trend over global communication rounds, where all six nodes demonstrate consistent error reduction when trained with HPO, highlighting its effectiveness in improving learning efficiency. Sub Fig. 11(b) compares the average MAE across all nodes per round, showing that models using HPO converge faster and maintain lower error rates throughout training. Sub Fig. 11(c) further confirms the benefits of HPO by presenting lower final MAE values per node compared to models without HPO. Additionally, sub Fig. 11(d) reports the AUC of the MAE curves, where HPO results in smaller AUC values, indicating more stable and accurate performance across the entire training process. These findings validate the utility of HPO in enhancing model accuracy and consistency, even in a symmetric and homogeneous ring topology.

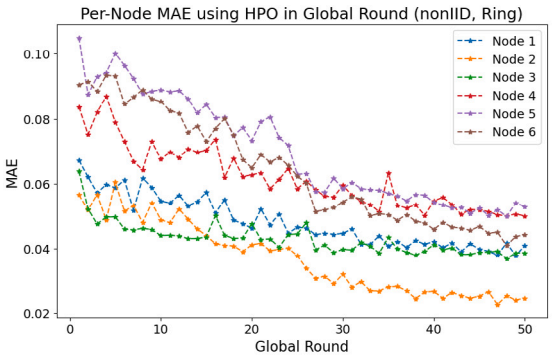
nonIID scenario: Fig. 12 demonstrates the impact of HPO under a non-IID data distribution and ring network topology for the Worker Safety dataset. As shown in Fig. 12(a), all six nodes experience a consistent reduction in MAE over global rounds when HPO is applied, indicating more stable and effective local learning. Fig. 12(b) reveals that the average MAE across rounds remains significantly lower with HPO, highlighting its role in improving generalization across the network despite the challenges of data heterogeneity. In Fig. 12(c), every node records a lower MAE with HPO compared to without, underscoring uniform performance gains. Most notably, Fig. 12(d) shows a drastic improvement in learning stability: the AUC of MAE is substantially lower with HPO, particularly on nodes 4 and 5, where the gap is largest. These results affirm the substantial benefits of localized hyperparameter optimization for reducing prediction error and increasing training reliability under non-IID conditions in ring topologies.

5.2.2. Performance evaluation using MNIST data

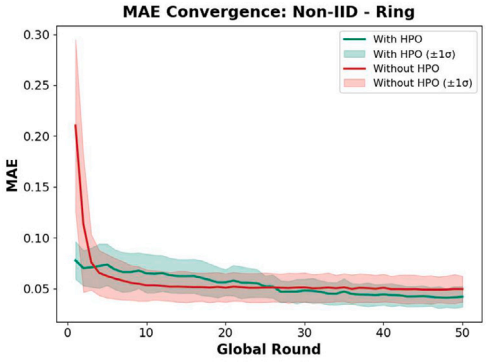
Analysis results in mesh topology.

IID scenario: Fig. 13 presents the comparative analysis of our decentralized federated learning framework with and without hyperparameter optimization (HPO) under an IID data distribution and mesh network topology. Fig. 13(a) shows that incorporating HPO consistently improves the average accuracy across global communication rounds, converging faster and achieving higher peak accuracy. Fig. 13(b) illustrates per-node accuracy, where each of the six nodes benefits from HPO with noticeable accuracy gains. Fig. 13(c) demonstrates that HPO reduces the number of rounds required to reach 90% accuracy on each node, indicating accelerated convergence. Lastly, Fig. 13(d) displays the area under the accuracy curve (AUC), further highlighting the stability and learning efficiency brought by HPO across all nodes. These results affirm the advantage of local hyperparameter tuning in decentralized training even in a homogeneous data setting.

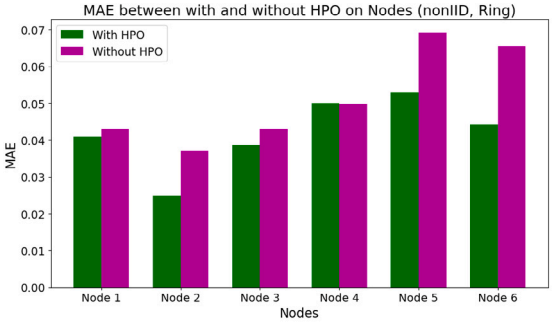
Non-IID scenario: Fig. 14 illustrates the performance of decentralized training with and without HPO under the non-IID data distribution in a mesh network topology. As shown in Fig. 14(a), models trained with HPO achieve significantly higher average accuracy across



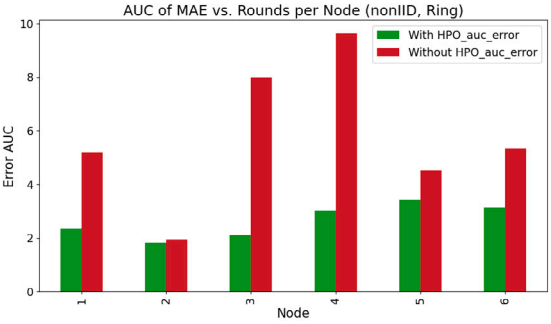
(a) Per node MAE across rounds using HPO in ring network



(b) Per round average MAE comparison in ring network: with vs without HPO



(c) Per node MAE comparison in ring network: with vs without HPO



(d) AUC of MAE per node comparison in ring network: with vs without HPO

Fig. 12. Decentralized training results under non-IID ring settings using the Worker Safety dataset: (a) per-node MAE across rounds using HPO, (b) average MAE per round comparing with and without HPO with shaded regions indicating $\pm 1\sigma$ variance across five independent runs, (c) final per-node MAE comparison, and (d) AUC of MAE per node comparison for learning stability assessment.

all communication rounds compared to those without HPO, indicating enhanced learning in heterogeneous data settings. Fig. 14(b) further confirms the improvement at the node level, where most nodes show markedly higher accuracy when HPO is applied. The convergence speed, as depicted in Fig. 14(c), is substantially better with HPO—nearly all nodes reach 90% accuracy in dramatically fewer rounds. Lastly, the AUC comparison in Fig. 14(d) reveals improved learning stability for nodes with HPO, as indicated by consistently higher AUC values. These results strongly support the effectiveness of local HPO in managing data heterogeneity and accelerating convergence in decentralized mesh networks.

Analysis results in ring topology.

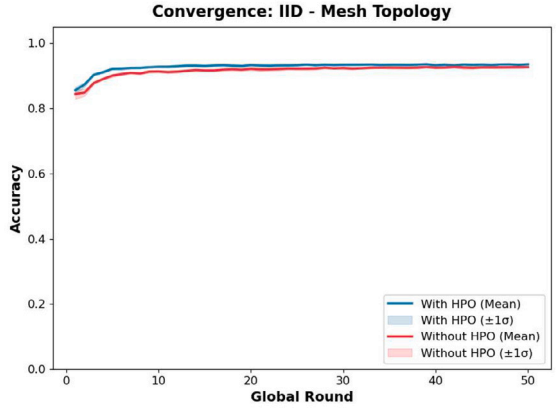
IID scenario: Fig. 15 presents the evaluation of decentralized training with and without HPO in the IID setting using a ring topology. As seen in Fig. 15(a), models trained with HPO exhibit both faster convergence and higher final accuracy across the communication rounds compared to those without HPO. This trend is further validated in Fig. 15(b), where all six nodes consistently achieve slightly better accuracy under HPO, indicating uniform improvement across the network. In Fig. 15(c), the number of global communication rounds required to reach 90% accuracy is significantly lower for all nodes when HPO is applied, demonstrating accelerated convergence. Lastly, Fig. 15(d) shows a consistently higher area under the accuracy curve (AUC) for all nodes with HPO, confirming improved learning stability throughout the training process. Collectively, these results reinforce the effectiveness of HPO in enhancing training efficiency, accuracy, and robustness even in topologies with constrained communication like the ring network.

Non-IID scenario: Fig. 16 presents the performance evaluation of decentralized training with and without HPO under the non-IID data distribution in a ring topology. As shown in Fig. 16(a), the model trained with HPO consistently outperforms the non-HPO counterpart across communication rounds, showing faster convergence and higher peak accuracy. The per-node performance in Fig. 16(b) indicates significant accuracy improvements across most nodes when HPO is applied, especially for those more affected by data heterogeneity. Fig. 16(c) reveals that the number of global rounds required to reach 90% accuracy is drastically reduced with HPO, reinforcing the advantage in convergence speed. Lastly, Fig. 16(d) demonstrates the stability of learning via the area under the curve (AUC), with HPO leading to better or comparable AUC values on all nodes. These findings confirm the effectiveness of local HPO in improving both efficiency and robustness of decentralized training in the challenging non-IID and ring-structured environment.

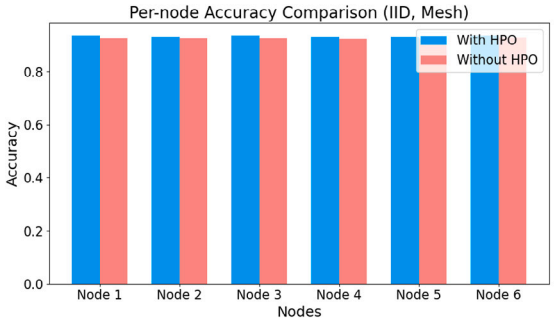
Across all evaluated conditions, Figs. 9–16 provide the round-wise performance trajectories, while Table 6 condenses representative final metrics and convergence statistics for direct baseline comparison.

5.3. Hardware resource and communication quantification

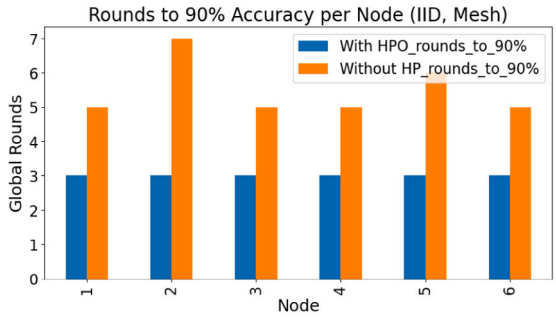
Table 6 provides a quantitative comparison of the proposed SMAC-based DFL-HPO against several baselines to evaluate its effectiveness and efficiency. For the worker-safety regression task, our method achieves a Mean Absolute Error (MAE) of 0.042, outperforming both the Default HPs (0.058) and Random Search (0.051), while reaching 90% convergence in significantly fewer rounds (18 vs. 42). A similar trend is observed in the MNIST classification task, where our approach attains the highest accuracy of 95.3% in only 3 rounds. Crucially, while Random Search requires an iterative communication overhead, our proposed SMAC-based mechanism utilizes a one-shot strategy, making it highly suitable for resource-constrained edge devices. The results also indicate that our decentralized framework closely approaches the



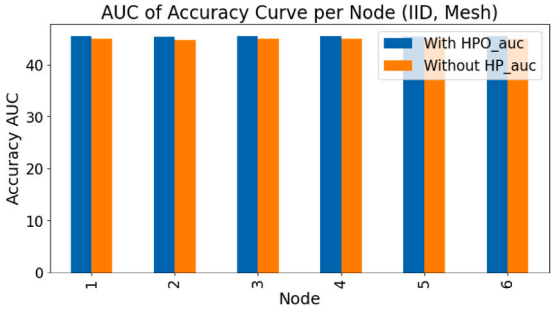
(a) Per round average accuracy comparison in mesh network: with vs without HPO



(b) Per node accuracy comparison in mesh network: with vs without HPO



(c) Round to 90% accuracy comparison in mesh network: with vs without HPO



(d) AUC per node comparison in mesh network: with vs without HPO

Fig. 13. Comparison of decentralized training with and without HPO under IID mesh settings: (a) accuracy over rounds with shaded regions indicating $\pm 1\sigma$ variance across five independent runs, (b) per-node accuracy, (c) convergence speed, and (d) learning stability via AUC.

Table 6
Quantitative performance comparison against standard HPO baselines.

Dataset	Method	Topology	Metric (MAE/Acc)	Rounds[a]	Comm. Type
Worker Safety	Default HPs	Mesh	0.058 (MAE)	42	N/A
Worker Safety	Random Search	Mesh	0.051 (MAE)	31	Iterative
Worker Safety	**Ours**	Mesh	**0.042 (MAE)**	**18**	**One-Shot**
MNIST	Default HPs	Mesh	92.4% (Acc)	8	N/A
MNIST	Random Search	Mesh	93.1% (Acc)	6	Iterative
MNIST	**Ours**	Mesh	**95.3% (Acc)**	**3**	**One-Shot**
Worker Safety	Centralized BO[b]	Star	0.039 (MAE)	12	Centralized

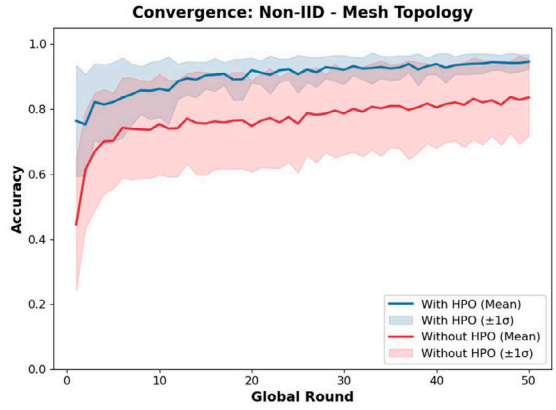
[a] Rounds to 90% convergence.
[b] Theoretical upper bound where all data is pooled at a central server.

theoretical upper bound represented by Centralized BO, which assumes full data pooling, a process often prohibited by privacy requirements in industrial settings.

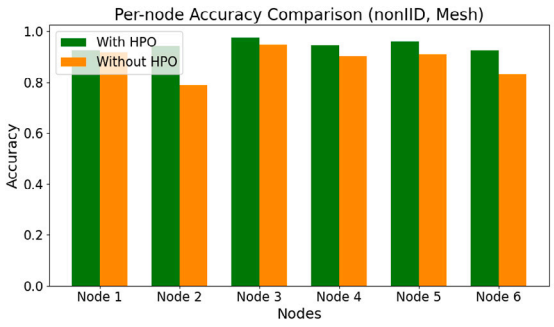
Quantitative summary of gains. Table 6 summarizes the representative quantitative benefit of the proposed HPO mechanism over the considered baselines. For the Worker Safety task, the proposed method reduced MAE from 0.058 to 0.042 relative to fixed hyperparameters and from 0.051 to 0.042 relative to Random Search, while reducing the rounds to target convergence from 42 to 18. For MNIST, the proposed method improved accuracy from 92.4% to 95.3% relative to fixed hyperparameters and from 93.1% to 95.3% relative to Random Search, while reducing the rounds to target convergence from 8 to 3. These results reinforce that the proposed one-shot SMAC-based HPO improves both final predictive performance and convergence efficiency under the same edge deployment setting.

To evaluate the practical feasibility of the proposed framework, we benchmarked the system on a Raspberry Pi 4 (4 GB RAM) testbed across three dimensions: computational time, memory footprint, and communication overhead, as summarized in Table 7. The approximate per-node one-shot HPO time was 12.5 min (IID mesh), 12.7 min (non-IID mesh), 15.0 min (IID ring), and 15.5 min (non-IID ring) for MNIST, and 5.0 min (IID mesh), 5.3 min (non-IID mesh), 6.5 min (IID ring), and 6.8 min (non-IID ring) for the Worker Safety dataset. These values indicate that the regression task incurs a substantially smaller tuning burden than the image-classification task, while the ring topology generally requires longer completion time due to its more constrained communication structure.

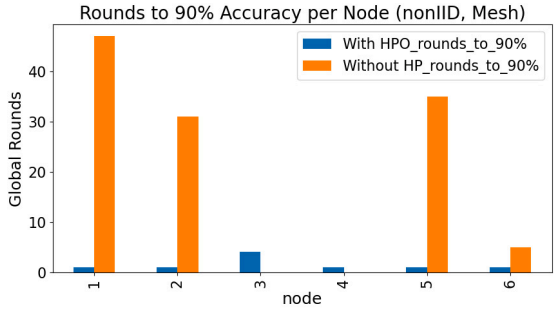
Rationale for baseline selection. Random Search was selected as the primary practical HPO baseline because it is lightweight, architecture-agnostic, and can be executed on resource-constrained edge devices under the same decentralized deployment constraints. By contrast, exhaustive Grid Search [25] becomes prohibitively expensive for mixed continuous/discrete/categorical search spaces, Hyperband [57] requires



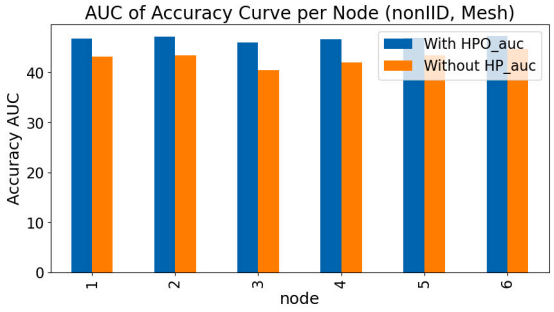
(a) Per round average accuracy comparison in mesh network: with vs without HPO



(b) Per node accuracy comparison in mesh network: with vs without HPO



(c) Round to 90% accuracy comparison in mesh network: with vs without HPO



(d) AUC per node comparison in mesh network: with vs without HPO

Fig. 14. Comparison of decentralized training with and without HPO under non-IID mesh settings: (a) accuracy over rounds with shaded regions indicating $\pm 1\sigma$ variance across five independent runs, (b) per-node accuracy, (c) convergence speed, and (d) learning stability via AUC.

Table 7
Hardware performance metrics: Raspberry Pi 4 (4GB RAM) Testbed.

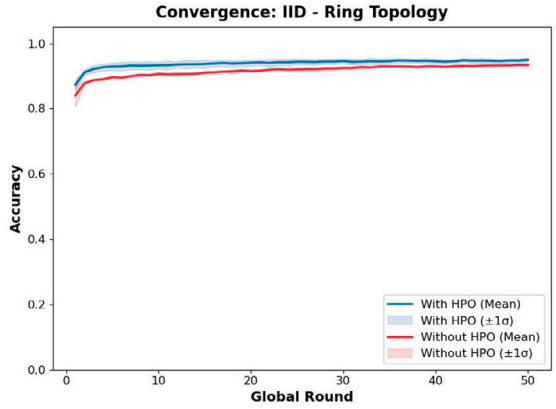
Topology	Metric	MNIST (Image)		Safety (Tabular)	
		IID	non-IID	IID	non-IID
Mesh	One-shot HPO Time	10–15 min	8–16 min	4–6 min	3–6.2 min
	Training (50 rds)	35–45 min	40–50 min	20–28 min	22–28 min
	Peak Memory Usage	2.8 GB	2.9 GB	1.8 GB	1.9 GB
Ring	One-shot HPO Time	12–18 min	10–20 min	5–8 min	4–9 min
	Training (50 rds)	42–55 min	45–60 min	25–32 min	28–35 min
	Peak Memory Usage	2.5 GB	2.6 GB	1.6 GB	1.7 GB

repeated budgeted partial-training schedules, and FedEx [29] is designed for centrally coordinated federated settings rather than the fully decentralized peer-to-peer framework studied here. We therefore consider Random Search the most appropriate edge-compatible comparison in the current revision, while broader baseline comparisons remain an important direction for future work.

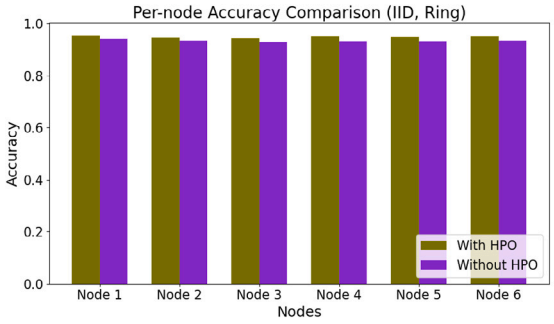
Computational, memory, and communication overhead. As shown in Table 7, the Mesh topology achieved shorter completion times than the Ring topology due to more direct peer-to-peer communication, although it incurred slightly higher peak memory usage because of more concurrent API connections. Even so, peak memory remained within the 4 GB device limit in all cases, ranging from 1.6–1.9 GB for the Worker Safety task and 2.5–2.9 GB for MNIST. To reduce computational burden, the one-shot HPO phase was executed on a 50% data subset, allowing tuning to complete in under 7 min for the tabular Safety dataset and under 16 min for MNIST in the mesh setting.

From the communication perspective, each HPO exchange contains only a compact metadata tuple comprising the incumbent hyperparameter configuration and its associated normalized validation performance, including fields such as learning rate, batch size, local epochs, optimizer, activation function, and loss value. In our implementation, this payload remains in the sub-kilobyte range in JSON form, whereas a compressed model update for the Worker Safety task is approximately 9 KB. Thus, even when forwarded during one-shot propagation, HPO exchange remains substantially lighter than repeated model transmission during DFL and accounts for less than 5% of the total network traffic.

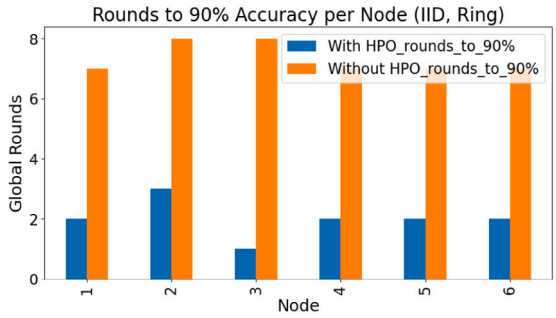
Regarding process-level overhead, DT-related services and local training were executed in separate processes, while DT rendering and visualization remained on the PC host. Therefore, the Raspberry Pi nodes carried only lightweight control-plane services in addition to local training and inference. This separation reduced resource contention on the edge devices and further supports the practical feasibility of the proposed framework.



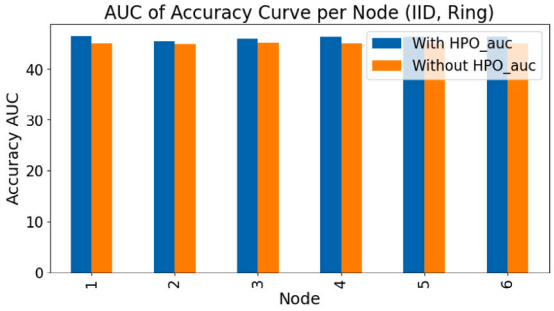
(a) Per round average accuracy comparison in mesh network: with vs without HPO



(b) Per node accuracy comparison in mesh network: with vs without HPO



(c) Round to 90% accuracy comparison in mesh network: with vs without HPO



(d) AUC per node comparison in mesh network: with vs without HPO

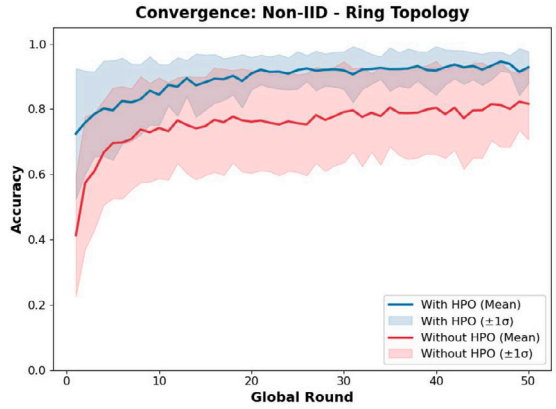
Fig. 15. Comparison of decentralized training with and without HPO under IID ring settings: (a) accuracy over rounds with shaded regions indicating $\pm 1\sigma$ variance across five independent runs, (b) per-node accuracy, (c) convergence speed, and (d) learning stability via AUC.

Convergence and topology trade-offs. The performance differences observed between the Mesh and Ring topologies can be further contextualized through the lens of graph theory and consensus optimization. In decentralized learning, the convergence rate is significantly influenced by the spectral gap (λ_2) of the communication graph, where a larger gap indicates better connectivity and faster information propagation. As shown in our results, the Mesh topology, possessing a superior spectral gap compared to the Ring, reached target accuracy in significantly fewer rounds. While more sophisticated weight-balancing schemes like Metropolis–Hastings or push-sum gossip could potentially accelerate convergence further, we opted for uniform neighbor averaging. This choice was a deliberate design decision to minimize the CPU and memory overhead on the Raspberry Pi nodes, which were already tasked with running the Orchestrator and the SMAC-based HPO phase. These results provide an empirical baseline for the DFL-HPO framework, and exploring advanced variance-reduction optimizers remains a promising avenue for future high-performance edge deployments.

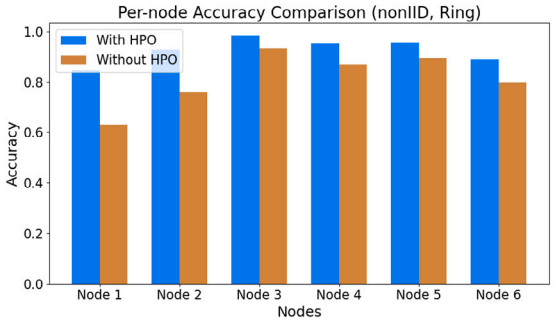
Trade-off between HPO overhead and convergence gain. A practical question is whether the additional cost of running local SMAC instances on resource-constrained edge devices is justified by the resulting performance gain. Our results indicate that the answer depends on the task complexity and deployment objective. On the Raspberry Pi 4 testbed, the one-shot HPO phase required approximately 3–9 min for the Worker Safety dataset and 8–20 min for MNIST, while the subsequent 50-round decentralized training phase required approximately 20–35 min and 35–60 min, respectively. Peak memory usage remained within the 4 GB device limit, ranging from 1.6–1.9 GB for the Safety task and 2.5–2.9 GB for MNIST. In return for this one-time tuning cost, the proposed method achieved consistently better predictive performance and faster convergence than fixed hyperparameter settings and standard Random Search, while exchanging only lightweight metadata during HPO. For example, in the representative mesh results reported in Table 6, the proposed method reduced the rounds to target convergence from 42 to 18 for the Worker Safety task and from 8 to 3 for MNIST, while also improving MAE and final accuracy. These results show that the front-loaded HPO cost is particularly favorable when rapid convergence, stable training, or repeated deployment under similar operating conditions is required.

Statistical validation. To determine if the performance improvements provided by the SMAC-based HPO are statistically significant, a one-tailed paired sample t-test was conducted on the final MAE results of the nodes ($N = 6$). The null hypothesis H_0 stated that there is no difference in mean error between the fixed-HP and HPO-enabled DFL. For the Worker Safety dataset, the results yielded $t(5) = 4.12$ and a p-value of 0.0046. Since $p < 0.01$, we reject H_0 and conclude with 99% confidence that the proposed cooperative HPO significantly reduces prediction error compared to standard decentralized learning.

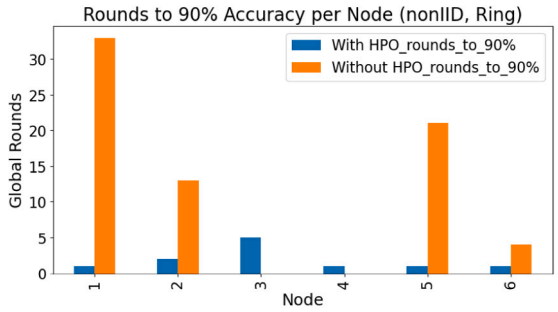
Robustness and variance quantification. To ensure the reproducibility and stability of the proposed framework, all experiments, including the SMAC-based HPO phase and the subsequent decentralized training, were repeated over five independent runs with different random seeds. The results presented in Figs. 9–16 depict the mean performance, while the shaded regions (or error bars) represent the standard deviation (σ) across these runs and the participating nodes. This approach accounts for the inherent stochasticity in the SMAC surrogate modeling, local SGD updates, and the random sampling associated with non-IID data distributions.



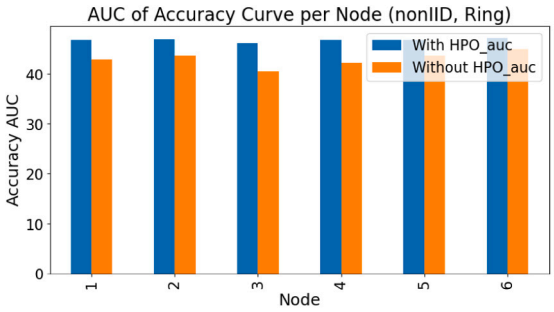
(a) Per round average accuracy comparison in mesh network: with vs without HPO



(b) Per node accuracy comparison in mesh network: with vs without HPO



(c) Round to 90% accuracy comparison in mesh network: with vs without HPO



(d) AUC per node comparison in mesh network: with vs without HPO

Fig. 16. Comparison of decentralized training with and without HPO under non-IID ring settings: (a) accuracy over rounds with shaded regions indicating $\pm 1\sigma$ variance across five independent runs, (b) per-node accuracy, (c) convergence speed, and (d) learning stability via AUC.

5.4. Ablation study

To evaluate the contribution of HPO to DFL performance, we conducted an ablation study comparing training outcomes with and without HPO across both image classification (MNIST) and regression (Worker Safety) tasks, under various network topologies (mesh and ring) and data distributions (IID and non-IID). The results consistently demonstrate that integrating local HPO substantially enhances model convergence speed, prediction accuracy, and training stability.

In the Worker Safety regression task, models trained with HPO achieved lower mean absolute error (MAE) across all nodes and communication rounds. Under IID mesh settings, HPO led to smoother and faster convergence, Figs. 9(a,b), lower final per-node MAEs, Fig. 9(c), and significantly improved learning stability, as evidenced by lower AUC metrics, Fig. 9(d). In the more challenging non-IID mesh scenario, the benefits were even more pronounced: HPO not only maintained its performance lead across nodes Figs. 10(a,c), but also accelerated early-stage learning Fig. 10(b) and produced more stable convergence patterns Fig. 10(d). These gains also persisted under a ring topology. In both IID and non-IID cases (Figs. 11, 12), HPO consistently reduced MAEs, improved inter-node consistency, and minimized training variance, despite the sequential communication limitations of ring structures. This performance gap between topologies aligns with theoretical expectations regarding the spectral gap of the network graph; the higher connectivity of the mesh topology facilitates faster information propagation, which the SMAC-based HPO leverages to achieve quicker consensus.

The MNIST classification task further supports the effectiveness of HPO in DFL. Under IID mesh conditions, models using HPO achieved higher average and per-node accuracy Figs. 13(a,b), reached 90% accuracy more quickly Fig. 13(c), and showed higher AUC values, indicating stable learning Fig. 13(d). When trained on non-IID data, HPO led to striking improvements: accuracy gains were substantial across nodes Fig. 14(b), convergence was accelerated Fig. 14(c), and learning remained stable despite data heterogeneity Fig. 14(d). Similar advantages were observed in the ring topology, where HPO improved final accuracy and convergence times under both IID and non-IID distributions (Figs. 15, 16).

Overall, this ablation confirms that including a local SMAC-based HPO module results in significant improvements across various learning tasks, topologies, and data distributions. Importantly, the computational overhead of the one-shot HPO phase (averaging 6-16 min as shown in Table 7) is offset by faster convergence, reducing the total time-to-target compared to fixed hyperparameters. The consistent reduction in error, faster convergence, and enhanced learning stability across all scenarios underscore the critical role of hyperparameter optimization in fully decentralized federated learning systems.

5.5. Practical implications of results

The experimental results presented in this work have direct and significant implications for deploying DFL systems in real-world edge environments, particularly in industrial safety-critical domains. The consistent performance gains observed across diverse datasets, network topologies, and data distributions demonstrate that integrating local HPO with decentralized learning pipelines is not merely a theoretical improvement; it is essential for ensuring model robustness, scalability, and responsiveness under real-world constraints.

First, the demonstrated reduction in prediction error and faster convergence using local HPO (via SMAC) means that on-device AI models can reach higher levels of accuracy in fewer communication

rounds. This directly translates to lower training latency, reduced energy consumption, and improved responsiveness, which are critical in industrial contexts where edge devices (e.g., wearables, sensors, embedded systems) operate under strict power and processing constraints. Our hardware benchmarking confirms that this is achievable on Raspberry Pi 4 nodes, with peak memory usage remaining below 2.9 GB, leaving ample headroom for the underlying OS and sensor drivers.

Second, the system's ability to maintain accuracy and stability under non-IID conditions and heterogeneous nodes reflects its robustness to real-world deployment scenarios, where uniform data distribution and synchronized resources are often unrealistic. In the context of occupational risk assessment, for example, different edge devices might observe varied working conditions, user behavior, and sensing quality. Our framework effectively adapts to such heterogeneity, making it suitable for decentralized safety analytics across large, diverse workforces without requiring centralized retraining or manual tuning.

Third, the performance benefits across both mesh and ring topologies indicate that the proposed approach is topology-agnostic, offering deployment flexibility for a wide range of network infrastructures from ad hoc mobile environments to structured sensor grids. This is particularly important in domains such as construction, logistics, or disaster response, where network connectivity and structure can change dynamically.

The integration of Orchestrator-driven virtualization significantly improves the observability and control of model training, hyperparameter tuning, and inference pipelines. The Orchestrator effectively separates the control plane from the data plane, ensuring that the management of the DT does not disrupt the critical inference cycles necessary for safety in high-stakes environments. This feature not only provides real-time feedback to operators and safety managers but also establishes a foundation for automated orchestration, debugging, and compliance monitoring—elements that are increasingly vital in regulated industrial and healthcare settings.

In summary, our findings highlight that deploying SMAC-driven HPO within DFL systems offers a practical, scalable, and privacy-preserving solution for real-time edge AI applications, particularly in environments where safety, accuracy, and reliability are paramount.

Implications for larger-scale edge-AI tasks. Although the current evaluation focuses on MNIST classification and worker-safety regression, the proposed framework is also relevant to larger-scale edge-AI workloads such as cross-domain remote-sensing segmentation, structural object detection, and crack detection/segmentation. Compared with the tasks studied in this paper, such applications typically involve denser outputs, larger model backbones, and higher communication and memory demands. Therefore, extending the present DFL-HPO framework to those settings would likely require budget-aware search spaces, model compression, hierarchical or sparse communication, and task-specific optimization objectives. We view these application domains as important next-step testbeds for assessing the scalability of decentralized hyperparameter optimization beyond lightweight classification and tabular regression.

Practical limitation of overhead analysis. Although the current study quantifies wall-clock time, memory footprint, and communication payload, we did not directly instrument energy consumption or fine-grained CPU utilization on the edge devices. Therefore, our feasibility discussion is based on measured execution time, peak memory usage, and relative communication burden rather than full power profiling. A more detailed energy-aware analysis, including per-process CPU usage and battery impact under continuous operation, remains an important direction for future work.

Adaptivity and concept drift mitigation. A potential limitation of the current one-shot HPO approach is its sensitivity to concept drift, particularly in dynamic industrial environments where sensor distributions may shift due to seasonal changes or new operational protocols. To ensure long-term model resilience, the Orchestrator can be configured to perform drift-triggered re-tuning. By leveraging the DT's real-time observability, the system can monitor the variance in local gradients and MAE. If performance degrades below a set reliability threshold, the Orchestrator initiates a localized HPO cycle using a sliding window of recent data. Our hardware analysis (Section 5.3) demonstrates that such retuning cycles would require only a brief 6-minute computational window, making dynamic adaptation a practical, low-overhead addition to the DFL pipeline.

6. Conclusions

Enhancing model performance in fully DFL environments remains a significant challenge due to the absence of a central server, non-IID data distributions, and heterogeneous device capabilities. This paper presents a robust HPO mechanism tailored for DFL, leveraging local SMAC-based tuners and a performance-weighted hyperparameter aggregation strategy. Our decentralized and cooperative HPO pipeline enables each node to autonomously adapt to its unique data and resource profile while still participating in a collaborative learning process without having a central server. By integrating this mechanism into a topology-flexible DFL framework and validating it on both public MNIST and private Worker Safety datasets using a Raspberry Pi testbed, we demonstrate substantial improvements in predictive accuracy, convergence speed, and learning stability. These benefits persist across both mesh and ring topologies, as well as under both IID and non-IID data conditions, underscoring the generalizability and robustness of our approach. Moreover, coupling this HPO pipeline with a DT layer provides real-time observability, configurability, and control, which are key features for deploying dependable AI at the edge in safety-critical scenarios. Our findings strongly suggest that decentralized HPO is not just beneficial but essential for achieving scalable and resilient performance in practical on-device AI networks. As such, the presented method lays the foundational work for DFL-HPO systems that are both resource-aware and performance-optimized.

Future research will focus on several key areas motivated by our experimental findings. First, we plan to implement dynamic hyperparameter scheduling to address potential concept drift. Our benchmarking results, which show that periodic re-tuning completes in less than 7 min, confirm that adaptive HPO is feasible for long-term deployments. Second, we aim to investigate advanced consensus mechanisms, such as Metropolis–Hastings weights, to improve convergence in ring topologies, which currently take longer than in mesh configurations (as detailed in Table 7). Furthermore, we intend to extend the SMAC tuner toward multi-objective optimization, balancing accuracy against latency and energy consumption to better align with real-world AIoT constraints. Finally, we will explore Byzantine-robust aggregation strategies, such as loss-clipping, to enhance system reliability in open-membership peer-to-peer environments beyond our current semi-trusted industrial setting.

CRediT authorship contribution statement

Sa Jim Soe Moe: Writing – review & editing, Writing – original draft, Methodology, Investigation, Data curation, Conceptualization. **Qazi Waqas Khan:** Writing – review & editing, Supervision, Resources, Conceptualization. **Anh Tuan Nguyen:** Validation, Resources. **Misbah Bibi:** Validation, Investigation. **Do Hyeun Kim:** Writing – review & editing, Supervision, Resources, Project administration, Funding acquisition, Data curation.

Declaration of competing interest

The author declares no conflict of interest.

Acknowledgments

This work was supported by Institute of Information & communications Technology Planning & evaluation (IITP) grant funded by the Korea government (MSIT) (No. RS-2025-02263031), and this research was supported by the Regional Innovation System & Education (RISE) program through the Jeju RISE center, funded by the Ministry of Education (MOE) and the Jeju Special Self-Governing Province, Republic of Korea (2026-RISE-17-001). Any correspondence related to this paper should be addressed to Do-Hyeun Kim.

Data availability

Data is available upon request.

References

[1] Brendan McMahan, Eider Moore, Daniel Ramage, Seth Hampson, Blaise Aguera y Arcas, Communication-efficient learning of deep networks from decentralized data, in: Artificial Intelligence and Statistics, PMLR, 2017, pp. 1273–1282.

[2] Tian Li, Anit Kumar Sahu, Ameet Talwalkar, Virginia Smith, Federated learning: Challenges, methods, and future directions, IEEE Signal Process. Mag. 37 (3) (2020) 50–60.

[3] Peter Kairouz, H. Brendan McMahan, Brendan Avent, Aurélien Bellet, Mehdi Bennis, Arjun Nitin Bhagoji, Kallista Bonawitz, Zachary Charles, Graham Cormode, Rachel Cummings, et al., Advances and open problems in federated learning, Found. Trends® Mach. Learn. 14 (1–2) (2021) 1–210.

[4] Jianyu Wang, Zachary Charles, Zheng Xu, Gauri Joshi, H. Brendan McMahan, Maruan Al-Shedivat, Galen Andrew, Salman Avestimehr, Katharine Daly, Deepesh Data, et al., A field guide to federated optimization, 2021, arXiv preprint arXiv:2107.06917.

[5] Tao Sun, Dongsheng Li, Bao Wang, Decentralized federated averaging, IEEE Trans. Pattern Anal. Mach. Intell. 45 (4) (2022) 4289–4301.

[6] Enrique Tomás Martínez Beltrán, Mario Quiles Pérez, Pedro Miguel Sánchez Sánchez, Sergio López Bernal, Gérôme Bovet, Manuel Gil Pérez, Gregorio Martínez Pérez, Alberto Huertas Celdrán, Decentralized federated learning: Fundamentals, state of the art, frameworks, trends, and challenges, IEEE Commun. Surv. & Tutorials 25 (4) (2023) 2983–3013.

[7] T.V. Nguyen, M.A. Dakka, S.M. Diakiw, M.D. VerMilyea, M. Perugini, J.M.M. Hall, D. Perugini, A novel decentralized federated learning approach to train on globally distributed, poor quality, and protected private medical data, Sci. Rep. 12 (1) (2022) 8888.

[8] Jiawen Kang, Dongdong Ye, Jiangtian Nie, Jiang Xiao, Xianjun Deng, Siming Wang, Zehui Xiong, Rong Yu, Dusit Niyato, Blockchain-based federated learning for industrial metaverses: Incentive scheme with optimal AoI, in: 2022 IEEE International Conference on Blockchain (Blockchain), IEEE, 2022, pp. 71–78.

[9] Jun Li, Yumeng Shao, Kang Wei, Ming Ding, Chuan Ma, Long Shi, Zhu Han, H. Vincent Poor, Blockchain assisted decentralized federated learning (BLADE-FL): Performance analysis and resource allocation, IEEE Trans. Parallel Distrib. Syst. 33 (10) (2021) 2401–2415.

[10] Lun Wang, Yang Xu, Hongli Xu, Min Chen, Liusheng Huang, Accelerating decentralized federated learning in heterogeneous edge computing, IEEE Trans. Mob. Comput. 22 (9) (2022) 5001–5016.

[11] Yuntao Wang, Zhou Su, Ning Zhang, Abderrahim Benslimane, Learning in the air: Secure federated learning for UAV-assisted crowdsensing, IEEE Trans. Netw. Sci. Eng. 8 (2) (2020) 1055–1069.

[12] Zhengxin Yu, Jia Hu, Geyong Min, Han Xu, Jed Mills, Proactive content caching for internet-of-vehicles based on peer-to-peer federated learning, in: 2020 IEEE 26th International Conference on Parallel and Distributed Systems, ICPADS, IEEE, 2020, pp. 601–608.

[13] Yuexiang Xie, Zhen Wang, Dawei Gao, Daoyuan Chen, Liuyi Yao, Weirui Kuang, Yaliang Li, Bolin Ding, Jingren Zhou, Federatedscope: A flexible federated learning platform for heterogeneity, 2022, arXiv preprint arXiv:2204.05011.

[14] Zhen Wang, Weirui Kuang, Ce Zhang, Bolin Ding, Yaliang Li, FedHPO-B: A benchmark suite for federated hyperparameter optimization, 2022, arXiv preprint arXiv:2206.03966.

[15] Abhijit Guha Roy, Shayan Siddiqui, Sebastian Pölsterl, Nassir Navab, Christian Wachinger, Braintorrent: A peer-to-peer environment for decentralized federated learning, 2019, arXiv preprint arXiv:1905.06731.

[16] Zhen Qin, Xueqiang Yan, Mengchu Zhou, Shuiguang Deng, BlockDFL: A blockchain-based fully decentralized peer-to-peer federated learning framework, in: Proceedings of the ACM Web Conference 2024, 2024, pp. 2914–2925.

[17] Yifan Shi, Li Shen, Kang Wei, Yan Sun, Bo Yuan, Xueqian Wang, Dacheng Tao, Improving the model consistency of decentralized federated learning, in: International Conference on Machine Learning, PMLR, 2023, pp. 31269–31291.

[18] Enrique Tomás Martínez Beltrán, Ángel Luis Perales Gómez, Chao Feng, Pedro Miguel Sánchez Sánchez, Sergio López Bernal, Gérôme Bovet, Manuel Gil Pérez, Gregorio Martínez Pérez, Alberto Huertas Celdrán, Fedstellar: A platform for decentralized federated learning, Expert Syst. Appl. 242 (2024) 122861.

[19] Salabat Khan, Fariha Nosheen, Syed Shehryar Ali Naqvi, Harun Jamil, Muhammad Faseeh, Murad Ali Khan, Do-Hyeun Kim, Bilevel hyperparameter optimization and neural architecture search for enhanced breast cancer detection in smart hospitals interconnected with decentralized federated learning environment, IEEE Access (2024).

[20] Anam Nawaz Khan, Qazi Waqas Khan, Atif Rizwan, Rashid Ahmad, Do Hyeun Kim, Consensus-driven hyperparameter optimization for accelerated model convergence in decentralized federated learning, Internet Things 30 (2025) 101476.

[21] Michael W. Grieves, Virtually intelligent product systems: Digital and physical twins, 2019.

[22] Obinna C. Madubuike, Chimay J. Anumba, Rana Khallaf, A review of digital twin applications in construction, J. Inf. Technol. Constr. 27 (2022).

[23] Calin Boje, Annie Guerriero, Sylvain Kubicki, Yacine Rezgui, Towards a semantic Construction Digital Twin: Directions for future research, Autom. Constr. 114 (2020) 103179.

[24] Marius Lindauer, Katharina Eggensperger, Matthias Feurer, André Biedenkapp, Difan Deng, Carolin Benjamins, Tim Ruhkopf, René Sass, Frank Hutter, SMAC3: A versatile Bayesian optimization package for hyperparameter optimization, J. Mach. Learn. Res. 23 (54) (2022) 1–9, URL http://jmlr.org/papers/v23/21-0888.html.

[25] James Bergstra, Rémi Bardenet, Yoshua Bengio, Balázs Kégl, Algorithms for hyper-parameter optimization, Adv. Neural Inf. Process. Syst. 24 (2011).

[26] James Bergstra, Yoshua Bengio, Random search for hyper-parameter optimization, J. Mach. Learn. Res. 13 (1) (2012) 281–305.

[27] Jasper Snoek, Hugo Larochelle, Ryan P. Adams, Practical bayesian optimization of machine learning algorithms, Adv. Neural Inf. Process. Syst. 25 (2012).

[28] Tzu-Ming Harry Hsu, Hang Qi, Matthew Brown, Measuring the effects of non-identical data distribution for federated visual classification, 2019, arXiv preprint arXiv:1909.06335.

[29] Zhongxiang Dai, Bryan Kian Hsiang Low, Patrick Jaillet, Federated Bayesian optimization via thompson sampling, Adv. Neural Inf. Process. Syst. 33 (2020) 9687–9699.

[30] Huanle Zhang, Mi Zhang, Xin Liu, Prasant Mohapatra, Michael DeLucia, Automatic tuning of federated learning hyper-parameters from system perspective, 2021.

[31] Yi Zhou, Parikshit Ram, Theodoros Salonidis, Nathalie Baracaldo, Horst Samulowitz, Heiko Ludwig, Flora: Single-shot hyper-parameter optimization for federated learning, 2021, arXiv preprint arXiv:2112.08524.

[32] Pengfei Guo, Dong Yang, Ali Hatamizadeh, An Xu, Ziyue Xu, Wenqi Li, Can Zhao, Daguang Xu, Stephanie Harmon, Evrim Turkbey, et al., Auto-fedrl: Federated hyperparameter optimization for multi-institutional medical image segmentation, in: European Conference on Computer Vision, Springer, 2022, pp. 437–455.

[33] Zhiyuan Li, Hao Li, Mingyang Zhang, Hyper-parameter tuning of federated learning based on particle swarm optimization, in: 2021 IEEE 7th International Conference on Cloud Computing and Intelligent Systems, CCIS, IEEE, 2021, pp. 99–103.

[34] Lun Wang, Yang Xu, Hongli Xu, Min Chen, Liusheng Huang, Accelerating decentralized federated learning in heterogeneous edge computing, IEEE Trans. Mob. Comput. 22 (2023) 5001–5016, URL https://api.semanticscholar.org/CorpusID:249144039.

[35] Pengfei Wang, Hao Yang, Guangjie Han, Ruiyun Yu, Leyou Yang, Geng Sun, Heng Qi, Xiaopeng Wei, Qiang Zhang, Decentralized navigation with heterogeneous federated reinforcement learning for UAV-enabled mobile edge computing, IEEE Trans. Mob. Comput. 23 (2024) 13621–13638, URL https://api.semanticscholar.org/CorpusID:271787725.

[36] Zhengxin Yu, Jia Hu, Geyong Min, Han Xu, Jed Mills, Proactive content caching for internet-of-vehicles based on peer-to-peer federated learning, in: 2020 IEEE 26th International Conference on Parallel and Distributed Systems, ICPADS, 2020, pp. 601–608, URL https://api.semanticscholar.org/CorpusID:232071790.

[37] Jiawen Kang, Dongdong Ye, Jiangtian Nie, Jiang Xiao, Xi Deng, Siming Wang, Zehui Xiong, Rong Yu, Dusit Tao Niyato, Blockchain-based federated learning for industrial metaverses: Incentive scheme with optimal AoI, in: 2022 IEEE International Conference on Blockchain (Blockchain), 2022, pp. 71–78, URL https://api.semanticscholar.org/CorpusID:249674602.

[38] Dongyu Chen, Tao Deng, Juncheng Jia, Siwei Feng, Di Yuan, Mobility-aware decentralized federated learning with joint optimization of local iteration and leader selection for vehicular networks, 2025, arXiv arXiv:2503.06443, URL https://api.semanticscholar.org/CorpusID:276903819.

[39] Qinglun Li, Li Shen, Guang-Ming Li, Quanjun Yin, Dacheng Tao, DFedADMM: Dual constraint controlled model inconsistency for decentralize federated learning, IEEE Trans. Pattern Anal. Mach. Intell. 47 (2025) 4803–4815, URL https://api.semanticscholar.org/CorpusID:276708687.

[40] Enrique Tom'as Mart'inez Beltr'an, Ángel Luis Perales Gómez, Chao Feng, Pedro Miguel S'anchez S'anchez, Sergio L'opez Bernal, Gérôme Bovet, Manuel Gil Pérez, Gregorio Mart'inez P'erez, Alberto Huertas Celdrán, Fedstellar: A platform for decentralized federated learning, Expert Syst. Appl. 242 (2023) 122861, URL https://api.semanticscholar.org/CorpusID:259187623.

[41] Xiaoyu Wang, Xuxing Chen, Shiqian Ma, Tong Zhang, Fully first-order methods for decentralized bilevel optimization, 2024, arXiv arXiv:2410.19319, URL https://api.semanticscholar.org/CorpusID:273638309.

[42] Salabat Khan, Fariha Nosheen, Syed Shehryar Ali Naqvi, Harun Jamil, Muhammad Faseeh, Murad Ali Khan, do-Hyeun Kim, Bilevel hyperparameter optimization and neural architecture search for enhanced breast cancer detection in smart hospitals interconnected with decentralized federated learning environment, IEEE Access 12 (2024) 63618–63628, URL https://api.semanticscholar.org/CorpusID:269347081.

[43] A. Khan, Qazi Waqas Khan, Atif Rizwan, Rashid Ahmad, DoHyeun Kim, Consensus-driven hyperparameter optimization for accelerated model convergence in decentralized federated learning, Internet Things 30 (2024) 101476, URL https://api.semanticscholar.org/CorpusID:275147344.

[44] Qibin He, Zhiyuan Yan, Wenhui Diao, Xian Sun, DLC: dynamic loss correction for cross-domain remotely sensed segmentation, IEEE Trans. Geosci. Remote Sens. 62 (2024) 1–14.

[45] Zicong Zhu, Xian Sun, Wenhui Diao, Kaiqiang Chen, Qibin He, Guangluan Xu, Kun Fu, CODet: Component object detector extracting structural features based on target characteristics, IEEE Trans. Geosci. Remote Sens. 61 (2023) 1–14.

[46] Hazrat Bilal, Mohammad S. Obaidat, Muhammad Shamrooz Aslam, Jing Zhang, Baoqun Yin, Khalid Mahmood, Online fault diagnosis of industrial robot using IoRT and hybrid deep learning techniques: An experimental approach, IEEE Internet Things J. 11 (19) (2024) 31422–31437.

[47] Rizwan Ullah, Hazrat Bilal, M.S. Aslam, Sarra Ayouni, Abdul Majid, Athanasios V. Vasilakos, Thippa Reddy Gadekallu, Conformer-PhyFaultNet: Physics-informed spectral attention conformer for generalizable bearing fault diagnosis, IEEE Internet Things J. (2026).

[48] Rizwan Ullah, Fazale Wahab, Hazrat Bilal, Sarra Ayouni, Gridsada Phanomchoeng, CrackFormer: A reinforcement learning-enhanced swin transformer V3 framework for high-fidelity detection and segmentation of pavement surface cracks, Alex. Eng. J. 131 (2025) 183–198.

[49] Zhiheng Zhao, Leidi Shen, Chen Yang, Wei Wu, Mengdi Zhang, George Q. Huang, IoT and digital twin enabled smart tracking for safety management, Comput. Oper. Res. 128 (2021) 105183, URL https://api.semanticscholar.org/CorpusID:230536645.

[50] Abiola Abosede Akanmu, Chimay J. Anumba, Omobolanle O. Ogunseiju, Towards next generation cyber-physical systems and digital twins for construction, J. Inf. Technol. Constr. 26 (2021) 505–525, URL https://api.semanticscholar.org/CorpusID:237247416.

[51] Sanjay Somanath, Vasilis Naserentin, Orfeas Eleftheriou, Daniel Sjölie, Beata Stahre Wästberg, Anders Logg, Towards urban digital twins: A workflow for procedural visualization using geospatial data, Remote. Sens. 16 (2024) 1939, URL https://api.semanticscholar.org/CorpusID:270109241.

[52] Qi Fang, Xiancong Chen, Daniel Castro-Lacouture, Chengqian Li, Intervention and management of construction workers' unsafe behavior: A simulation digital twin model, Adv. Eng. Inform. 58 (2023) 102182, URL https://api.semanticscholar.org/CorpusID:261849532.

[53] Rafael Sacks, Ioannis K. Brilakis, Ergo Pikas, Haiyan Xie, Mark A. Girolami, Construction with digital twin information systems, Data-Centric Eng. 1 (2020) URL https://api.semanticscholar.org/CorpusID:229670088.

[54] Wen Sun, Shiyu Lei, Lu Wang, Zhiqiang Liu, Yan Zhang, Adaptive federated learning and digital twin for industrial internet of things, IEEE Trans. Ind. Inform. 17 (2020) 5605–5614, URL https://api.semanticscholar.org/CorpusID:225067768.

[55] Udayakumar Kamalakannan, Ramamoorthy Sriramulu, Poorvadevi Ramamurthi, DTFL-DF: Digital twin architecture powered by federated learning decision forest to mitigate fire accidents in mining industry, Syst. Eng. 27 (2024) 869–885, URL https://api.semanticscholar.org/CorpusID:268910677.

[56] Lun Tang, Mingyan Wen, Zhenzhen Shan, Li Li, Qinghai Liu, Qianbin Chen, Digital twin-enabled efficient federated learning for collision warning in intelligent driving, IEEE Trans. Intell. Transp. Syst. 25 (2024) 2573–2585, URL https://api.semanticscholar.org/CorpusID:266492821.

[57] Lisha Li, Kevin Jamieson, Giulia DeSalvo, Afshin Rostamizadeh, Ameet Talwalkar, Hyperband: A novel bandit-based approach to hyperparameter optimization, J. Mach. Learn. Res. 18 (185) (2018) 1–52.